

Apêndice A

Projetos para ensinar organização e arquitetura de computadores

- A.1** Simulações interativas
- A.2** Projetos de pesquisa
- A.3** Projetos de simulação
 - SimpleScalar
 - SMPCache
- A.4** Projetos da linguagem de montagem
- A.5** Atividades de leitura/relatórios
- A.6** Atividades de escrita
- A.7** Banco de testes

Muitos professores acreditam que as pesquisas ou os projetos de implementação sejam cruciais para o claro entendimento dos conceitos da organização e arquitetura de computadores. Sem os projetos, pode ser difícil para os estudantes compreenderem alguns conceitos básicos e as interações entre os componentes. Os projetos reforçam os conceitos introduzidos no livro, dão aos estudantes uma compreensão melhor do trabalho interno dos processadores e dos sistemas computacionais e podem motivar os estudantes e dar-lhes a confiança de que dominaram a matéria.

Neste texto tentei apresentar os conceitos da arquitetura e organização de computadores da forma mais clara possível e forneci vários problemas para trabalhos de casa para reforçar esses conceitos. Muitos professores

desejarão complementar este material com projetos. Este apêndice fornece uma orientação para esse fim e descreve o material de suporte disponível no manual do professor. O material de suporte cobre seis tipos de projetos e outros exercícios para estudantes:

- Simulações interativas.
- Projetos de pesquisa.
- Projetos de simulação.
- Projetos da linguagem de montagem.
- Atividades de leitura/relatórios.
- Atividades de escrita.
- Banco de testes.



A.1 Simulações interativas

A novidade desta edição é a incorporação de simulações interativas. Elas fornecem uma ferramenta poderosa para o entendimento dos recursos de um projeto complexo de um sistema computacional moderno. Os estudantes de hoje querem ser capazes de visualizar vários mecanismos de sistemas de computadores complexos nas telas dos seus próprios computadores. Um total de 200 simulações é usado para ilustrar as funções principais e os algoritmos na organização e arquitetura de computadores. A Tabela A.1 lista as simulações por capítulo. Em pontos relevantes deste livro, um ícone indica que uma simulação interativa relevante está disponível online para uso dos estudantes.

Tabela A.1 Organização e arquitetura do computador — Simulações interativas por capítulo

Capítulo 4 — Memória cache	
Simulador de cache	Emula caches pequenas com base em um simples modelo de cache inserido pelo usuário e mostra o conteúdo da cache no final do ciclo simulado, baseado na inserção de uma sequência de escrita feita pelo usuário, ou gerada randomicamente, se selecionada.
Análise de tempo da cache	Demonstra a média de tempo de acesso à memória para os parâmetros de cache que você especificar.
Simulador de cache multitarefa	Modela a cache em um sistema que suporta multitarefa.
Simulador de cache Selective victim	Compara três políticas de cache diferentes.
Capítulo 5 — Memória interna	
Simulador de memória intercalada	Demonstra o efeito de memória intercalada.
Capítulo 6 — Memória externa	
RAID	Determina a eficiência e a confiabilidade de armazenamento.
Capítulo 7 — Entrada/Saída	
Sistema e ferramenta de projeto de E/S	Avalia custos e desempenhos comparativos de diferentes sistemas de E/S.
Capítulo 8 — Suporte do SO	
Algoritmos de substituição de página	Compara LRU, FIFO e Optimal.
Mais algoritmos de substituição de página	Compara várias políticas.
Capítulo 12 — Estrutura e funcionamento da CPU	
Analizador de tabela de reserva	Avalia tabelas de reserva, que é um meio de representar o modelo do fluxo de para um sistema pipeline.
Branch Prediction	Demonstra três diferentes esquemas de previsão de desvio.
Branch Target Buffer	Simulador que combina branch predictor/branch target buffer.
Capítulo 13 — Computadores com conjunto de instruções reduzido (RISC)	
Política de estado — MIPS 5	Simula o pipeline.
Desdobramento do Loop	Simula a técnica de desdobramento do loop para explorar o paralelismo no nível de instruções.
Capítulo 14 — Paralelismo em nível de instruções e processadores superescalares	
Pipeline com agendamento estático <i>versus</i> dinâmico	Uma simulação mais complexa do pipeline MIPS.
Simulador de reordenação de buffer	Simula reordenação de instrução em um pipeline RISC.
Técnica para simulador de scoreboarding dinâmico	Simulação de uma técnica de escalonamento de instrução usada em um número de processadores.
Algoritmo de Tomasulo	Simulação de outra técnica de escalonamento de instrução.
Simulação alternativa do algoritmo de Tomasulo	Outra simulação do algoritmo de Tomasulo.
Capítulo 17 — Organização paralela	
Simulação de processamento vetorial	Demonstra a execução de instruções de processamento vetorial.

Como as simulações possibilitam que o usuário defina condições iniciais, elas podem servir como base para as tarefas dos estudantes.

As simulações interativas foram desenvolvidas sob a direção do professor Israel Koren, da *University of Massachusetts Department of Electrical and Computer Engineering*. Aswin Sreedhar, da *University of Massachusetts* desenvolveu atividades de simulação interativa.



A.2 Projetos de pesquisa

Uma forma eficiente de reforçar os conceitos básicos do curso e de ensinar aos alunos habilidades de pesquisa é atribuir-lhes um projeto de pesquisa. Tal projeto pode envolver uma pesquisa de literatura assim como uma pesquisa pela Internet dos produtos dos fabricantes, atividades nos laboratórios de pesquisa e esforços para padronização. Os projetos podem ser atribuídos para equipes ou, para projetos menores, para indivíduos. Em todo caso, o melhor é requerer algum tipo de proposta de projeto logo cedo, para que o professor tenha tempo de avaliar a proposta para determinado assunto e o nível de esforço adequado. O folheto dos alunos para projetos de pesquisa deve conter:

- Um formato para proposta.
- Um formato para relatório final.
- Uma agenda com prazos intermediários e finais.
- Uma lista de possíveis assuntos do projeto.

Os estudantes podem selecionar um dos assuntos relacionados ou inventar seus próprios projetos comparáveis.



A.3 Projetos de simulação

Uma maneira excelente de obter a compreensão da operação interna do processador e para estudar e apreciar algumas negociações de projeto e implicações de desempenho é simular os principais elementos do processador. Duas ferramentas úteis para este propósito são o SimpleScalar e o SMPCache.

Comparada à implementação de hardware real, a simulação oferece duas vantagens para uso educacional e de pesquisa:

- Com a simulação, fica fácil modificar vários elementos de uma organização, variar as características de diferentes elementos e depois analisar os efeitos de tais modificações.
- A simulação oferece uma coleção de estatísticas detalhadas de desempenho, que pode ser usado para entender as relações da desempenho.



SimpleScalar

O SimpleScalar (BURGER e AUSTIN, 1997^a; MANJIKIAN, 2001^b; MANJIKIAN, 2001^c) é um conjunto de ferramentas que pode ser usado para simular programas reais em vários processadores e sistemas modernos. O conjunto de ferramentas inclui compilador, *assembler*, *linker* e ferramentas de simulação e visualização. Ele fornece simuladores de processadores que variam de um simulador funcional extremamente rápido até um simulador detalhado de processador superescalar com envio fora de ordem, que suporta caches sem bloqueio e execução especulativa. A arquitetura do conjunto de instruções e os parâmetros organizacionais podem ser modificados para criar uma variedade de experimentos.

O SimpleScalar é um pacote de software portátil que executa na maioria de plataformas UNIX. O software SimpleScalar pode ser debatido no site de SimpleScalar. Está disponível sem custo para uso não comercial.



SMPCache

O SMPCache é um simulador orientado a rastreamento para análise e ensino de sistemas de memória cache em multiprocessadores simétricos (RODRIGUEZ, PEREZ e PULIDO, 2001^d). A simulação é baseada em um modelo construído de acordo com os princípios básicos de arquitetura desses sistemas. O simulador possui uma interface gráfica completa e amigável. Alguns dos parâmetros que podem ser estudados com o simulador são: localidade do programa, influência do número de processadores, protocolos de coerência de cache, esquemas para arbitragem do barramento, mapeamento, políticas de substituição, tamanho da cache (blocos na cache), número de conjuntos de cache (para conjuntos de caches associativas) e número de palavras por bloco (tamanho do bloco de memória).

O SMPCache é um pacote de software portátil que executa em sistemas PC com Windows. Ele pode ser obtido no site de SMPCache e está disponível sem custo para uso não comercial.



A.4 Projetos da linguagem de montagem

A programação na linguagem de montagem é usada para ensinar aos alunos os componentes de hardware de baixo nível e os fundamentos da arquitetura computacional. O CodeBlue é um programa simplificado para linguagem de montagem desenvolvido pela *Academia de Força Aérea Americana*. O objetivo do trabalho foi desenvolver e ensinar os conceitos da linguagem de montagem usando um simulador visual que permita que os estudantes aprendam em uma única aula. Os desenvolvedores quiseram também que os alunos achassem a linguagem motivadora e divertida de usar. A linguagem CodeBlue é muito mais simples do que a maioria de conjuntos de instruções de arquitetura simplificados como SC123. Mesmo assim, ela permite que alunos desenvolvam programas *assembly* interessantes que competem em torneios, da forma semelhante ao simulador SPIMbot – muito mais complexo. Mais importante, por meio da programação com CodeBlue, os estudantes aprendem os conceitos básicos da arquitetura computacional como coexistência de instruções e dados em memória, implementação da estrutura de controle e modos de endereçamento.

Para fornecer uma base para projetos, os desenvolvedores construíram um ambiente visual de desenvolvimento que permite que alunos criem um programa, vejam a sua representação em memória, passem pela execução do programa e simulem uma batalha de programas que competem em um ambiente de memória visual.

Os projetos podem ser construídos em cima do conceito de um torneio de Guerra de Núcleo (Core War). O Guerra de Núcleo é um jogo de programação introduzido para o público no começo dos anos 1980 e que foi popular durante um período de mais ou menos 15 anos. Ele possui quatro componentes principais: uma matriz de memória de 8.000 endereços, uma linguagem de montagem simplificada Redcode, um programa executável chamado MARS (um acrônimo para *Memory Array Redcode Simulator* – Simulador Redcode para Matriz de Memória) e o conjunto de programas de batalha que competem. Dois programas de batalhas são inseridos na matriz de memória em posições aleatórias; nenhum programa sabe onde o outro está. O MARS executa os programas em uma versão simples de tempo compartilhado. Os dois programas se alternam: uma única instrução do primeiro programa é executada, depois uma do segundo e assim por diante. O que um programa de batalha faz durante os ciclos de execução a ele atribuídos depende inteiramente do programador. O objetivo é destruir o outro programa arruinando as suas instruções. O ambiente CodeBlue substitui CodeBlue para Redcode e fornece a sua própria interface interativa de execução.



A.5 Atividades de leitura/relatórios

Outra maneira excelente para reforçar os conceitos do curso e dar aos estudantes a experiência em pesquisa é atribuir artigos da literatura para serem lidos e analisados. O site deste livro inclui uma lista sugerida de artigos que podem ser utilizados para atividades de leitura e relatórios.

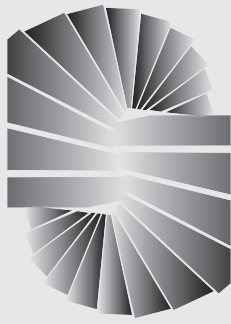


A.6 Atividades de escrita

Atividades de escrita podem ter um eficiente efeito multiplicador no processo de aprendizagem em uma disciplina técnica como comunicação de dados e redes. Adeptos do movimento Writing Across the Curriculum (WAC — Escrever Através do Currículo: <<http://wac.colostate.edu/>>) reportam benefícios substanciais de atividades de escrita para facilitar a aprendizagem. Elas levam ao pensamento mais detalhado e completo sobre um determinado assunto. Além disso, atividades de escrita ajudam a superar a tendência dos alunos de seguir um assunto com o mínimo de engajamento pessoal, apenas aprendendo fatos e técnicas de resolução de problemas sem obter um entendimento profundo do assunto.

Referências

- a BURGER, D. e AUSTIN, T. "The SimpleScalar tool set, version 2.0". *Computer Architecture News*, jun. 1997.
- b MANJIKIAN, N. "More enhancements of the SimpleScalar tool set". *Computer Architecture News*, set. 2001.
- c MANJIKIAN, N. "Multiprocessor enhancements of the SimpleScalar tool set". *Computer Architecture News*, mar. 2001.
- d RODRIGUEZ, M.; PEREZ, J. e PULIDO, J. "An educational tool for testing caches on symmetric multiprocessors". *Microprocessors and Microsystems*, jun. 2001.



Apêndice B

Linguagem de montagem (*assembly*) e assuntos relacionados

B.1 Linguagem de montagem

- Elementos da linguagem de montagem
- Tipos de setenças da linguagem de montagem
- Exemplo: programa do maior divisor comum

B.2 Montadores

- Montador de dois passos
- Montador de um passo
- Exemplo: programa de números primos

B.3 Carregamento e ligação

- Realocação
- Carregamento
- Ligação

B.4 Leitura recomendada e sites Web

- Sites web recomendados

PRINCIPAIS PONTOS

- Uma linguagem de montagem (*assembly*) é uma representação simbólica da linguagem de máquina de um processador específico, acrescida de tipos de instruções adicionais que facilitam a escrita do programa e que fornecem as instruções para o montador (*assembler*).
- Um montador é um programa que traduz a linguagem de montagem em código de máquina.
- O primeiro passo na criação de um processo ativo é carregar um programa na memória principal e criar uma imagem do processo.
- Um linker (ligador) é usado para resolver quaisquer referências entre os módulos carregados.

O assunto sobre a linguagem de montagem foi introduzido brevemente no Capítulo 11. Este apêndice fornece mais detalhes e cobre também uma série de assuntos relacionados. Há várias razões para estudar a linguagem de programação de montagem (quando comparada com uma linguagem de programação de alto nível), incluindo o seguinte:

1. Ela esclarece a execução de instruções.
2. Ela mostra como os dados são representados na memória.
3. Ela mostra como um programa interage com o sistema operacional, o processador e o sistema de E/S.
4. Ela esclarece como um programa acessa dispositivos externos.
5. Permite entender programadores de linguagem de montagem, fazendo com que os alunos se tornem programadores melhores de linguagens de programação de alto nível (HLL), dando-lhes uma ideia melhor da linguagem alvo para a qual a HLL deve ser traduzida.

Começamos este apêndice com um estudo de elementos básicos de uma linguagem de montagem, usando arquitetura x86 para nossos exemplos.¹ A seguir, analisamos a operação do montador. Isso é seguido por uma discussão de linkers e carregadores (loaders).

A Tabela B.1 define alguns dos principais termos usados neste apêndice.

¹ Existem vários montadores para arquitetura x86. Nossos exemplos usam NASM (*Netwide Assembler*), um montador de código fonte aberto. Uma cópia do manual de NASM está no site Web deste livro.

Tabela B.1 Principais termos para este apêndice

Montador
Um programa que traduz a linguagem de montagem para código de máquina.
Linguagem de montagem (Linguagem de montagem)
Uma representação simbólica da linguagem de máquina de um processador específico, acrescida de tipos de instruções adicionais que facilitam a escrita do programa e que fornecem instruções para o montador.
Compilador
Um programa que converte outro programa de alguma linguagem fonte (ou linguagem de programação) para linguagem de máquina (código objeto). Alguns compiladores geram saída em linguagem de montagem que é então convertida para linguagem de máquina por um montador diferente. Um compilador se distingue de um montador pelo fato de que cada instrução de entrada, em geral, não corresponde a uma única instrução de máquina ou uma sequência fixa de instruções. Um compilador pode suportar recursos como alocação automática de variáveis, expressões aritméticas arbitrárias, estruturas de controle de laços como FOR e WHILE, escopo de variável, operações de entrada/saída, funções de alto nível e portabilidade de código fonte.
Código executável
O código de máquina gerado por um processador da linguagem de código fonte como um montador ou um compilador, isto é, software em uma forma que pode ser executada no computador.
Conjunto de instruções
O conjunto de todas as instruções possíveis para um determinado computador, isto é, conjunto de instruções de linguagem de máquina que um determinado processador entende.
Linker (Ligador)
Um programa utilitário que combina um ou mais arquivos contendo código objeto de módulos de programa compilados separadamente para um arquivo único contendo código carregável ou executável.
Loader (Carregador)
Uma rotina de programa que carrega um programa executável na memória para execução.
Linguagem de máquina ou código de máquina
Representação binária de um programa de computador que é lido e interpretado de fato pelo computador. Um programa em código de máquina consiste de uma sequência de instruções de máquina (possivelmente intercaladas com dados). Instruções são cadeias binárias que podem ser todas do mesmo tamanho (por exemplo, uma palavra de 32 bits para muitos microprocessadores RISC modernos) ou de tamanhos diferentes.
Código objeto
Representação, em linguagem de máquina, do código fonte de programação. O código objeto é criado por um compilador ou montador e é transformado em código executável pelo linker.



B.1 Linguagem de montagem

A linguagem de montagem é uma linguagem de programação que está a um passo de distância da linguagem de máquina. Normalmente, cada instrução da linguagem de montagem é traduzida em uma instrução de máquina pelo montador. A linguagem de montagem é dependente do hardware, com uma linguagem de montagem diferente para cada tipo de processador. Em particular, as instruções da linguagem de montagem devem fazer referência aos registradores específicos do processador, incluir todos os *opcodes* do processador e refletir o tamanho em bits de vários registradores do processador e dos operandos da linguagem de máquina. Portanto, um programador de linguagem de montagem deve compreender a arquitetura do computador.

Programadores raramente usam linguagem de montagem para aplicações ou até programas de sistemas. Linguagens de programação de alto nível fornecem capacidades expressivas de concisões que facilitam consideravelmente as tarefas de programação. As desvantagens de usar uma linguagem de montagem em vez de linguagem HLL são (FOG, 2008³):

1. **Tempo de desenvolvimento.** Escrever código em linguagem de montagem leva muito mais tempo do que escrever em uma linguagem de alto nível.

2. **Confiabilidade e segurança.** É fácil cometer erros no código em linguagem de montagem. O montador não verifica se as convenções de chamada e convenções para salvar registradores são obedecidas. Ninguém verifica se o número de instruções PUSH e POP é o mesmo em todos os desvios e caminhos possíveis. Existem tantas possibilidades para erros escondidos em código em linguagem de montagem que isso afeta a confiabilidade e a segurança do projeto, a não ser que se tenha uma abordagem muito sistemática para testes e verificações.
3. **Depuração e verificação.** O código em linguagem de montagem é mais difícil de depurar e verificar porque há mais possibilidades para erros do que em código de alto nível.
4. **Manutenção.** O código em linguagem de montagem é mais difícil de modificar e manter porque a linguagem permite código "espaguete" não estruturado e todo tipo de truques que são difíceis de serem entendidos por outras pessoas. Documentação minuciosa e um estilo de programação consistente são necessários.
5. **Portabilidade.** O código em linguagem de montagem é específico para plataforma. Portar para uma plataforma diferente é difícil.
6. **Código de sistemas podem usar funções intrínsecas em vez de linguagem de montagem.** Os melhores compiladores modernos para C++ possuem funções intrínsecas para acessar registradores de controle do sistema e outras instruções do sistema. O código em linguagem de montagem não é mais necessário para drivers de dispositivos e outros códigos do sistema quando funções intrínsecas estão disponíveis.
7. **Código da aplicação pode usar funções intrínsecas ou classes vetoriais em vez de linguagem de montagem.** Os melhores compiladores modernos para C++ possuem funções intrínsecas para operações vetoriais e outras instruções especiais que antes requeriam programação em linguagem de montagem.
8. **Compiladores melhoraram muito nos últimos anos.** Os melhores compiladores atualmente são muito bons. É necessário ter muita perícia e experiência para otimizar melhor do que o melhor compilador C++.

Mesmo assim, ainda existem algumas vantagens para uso ocasional de linguagem de montagem, incluindo o seguinte (FOG, 2008^a):

1. **Depuração e verificação.** Analisar código em linguagem de montagem gerado pelo compilador ou olhar a janela de montagem em um depurador é útil para localizar erros e verificar a qualidade da otimização feita pelo compilador para um determinado pedaço de código.
2. **Desenvolver compiladores.** Entender técnicas de codificação em linguagem de montagem é necessário para criar compiladores, depuradores e outras ferramentas de desenvolvimento.
3. **Sistemas embarcados.** Sistemas embarcados pequenos possuem menos recursos do que PCs e mainframes. A programação em linguagem de montagem pode ser necessária para otimizar o código em velocidade ou em tamanho em sistemas embarcados pequenos.
4. **Drivers para hardware e códigos de sistemas.** Acessar hardware, registradores de controle do sistema e etc. às vezes pode ser difícil ou impossível com código de alto nível.
5. **Acessar instruções que não são acessíveis a partir das linguagens de alto nível.** Certas instruções em linguagem de montagem não possuem um equivalente na linguagem de alto nível.
6. **Código que se modifica por si.** O código que se modifica por si normalmente não é lucrativo porque interfere com um código eficiente para uso de cache. No entanto, ele pode ser vantajoso, por exemplo, para incluir um pequeno compilador em programas matemáticos, onde uma função definida pelo usuário precisa ser calculada várias vezes.
7. **Otimizar o tamanho do código.** O espaço de armazenamento e a memória são tão baratos hoje em dia que não vale a pena usar linguagem de montagem para reduzir o tamanho do código. No entanto, o tamanho da cache ainda é um recurso crítico que pode se tornar útil em alguns casos e otimizar o tamanho de um pedaço de código para que caiba dentro da cache de código.
8. **Otimizar velocidade do código.** Compiladores C++ modernos geralmente otimizam o código muito bem na maioria dos casos. Mas ainda há situações nas quais os compiladores são fracos e aumentos dramáticos de velocidade podem ser alcançados por meio de uma cuidadosa programação em linguagem de montagem.
9. **Bibliotecas de funções.** O benefício total de otimizar código é maior em bibliotecas de funções que são usadas por muitos programadores.
10. **Tornar bibliotecas de funções compatíveis com vários compiladores e sistemas operacionais.** É possível criar funções de biblioteca com várias entradas que sejam compatíveis com diferentes compiladores e diferentes sistemas operacionais. Isso requer programação em linguagem de montagem.

Os termos *linguagem de montagem* e *linguagem de máquina* às vezes são usados, erroneamente, como sinônimos. A linguagem de máquina consiste de instruções executadas diretamente pelo processador. Cada instrução de linguagem de máquina é uma cadeia binária contendo um *opcode*, referências a operandos e talvez outros bits relacionados à execução, como flags. Por conveniência, em vez de escrever uma instrução com uma cadeia de bits, ela pode ser escrita simbolicamente, com nomes para *opcodes* e registradores. Uma linguagem de montagem faz uso muito maior de nomes simbólicos, incluindo atribuição de nomes a posições específicas da memória principal e posições específicas das instruções. Ela inclui também instruções que não são executadas diretamente, mas servem como instruções para o montador que produz código de máquina a partir de um programa na linguagem de montagem.



Elementos da linguagem de montagem

Uma sentença em uma linguagem de montagem típica tem a forma mostrada na Figura B.1. Ela consiste de quatro elementos: rótulo, mnemônico, operando e comentário.

RÓTULO Se um rótulo está presente, o montador define o rótulo como equivalente ao endereço no qual o primeiro byte do código objeto gerado para essa instrução será carregado. O programador pode usar o rótulo subsequentemente como um endereço ou como dados no campo de endereço de outra instrução. O montador substitui o rótulo com o valor atribuído quando cria o programa objeto. Rótulos são usados com mais frequência em instruções de desvio.

Como um exemplo, aqui está uma parte de um programa:

```
L2: SUB    EAX, EDX ; subtrai conteúdo do registrador EDX do conteúdo
                        ; de EAX e armazena o resultado em EAX
      JG    L2      ; salta para L2 se resultado da subtração for positivo
```

O programa continuará no laço de volta para posição L2 até que o resultado seja zero ou negativo. Desta forma, quando a instrução `kg` é executada, se o resultado é positivo, o processador coloca o endereço equivalente ao rótulo L2 no contador de programa.

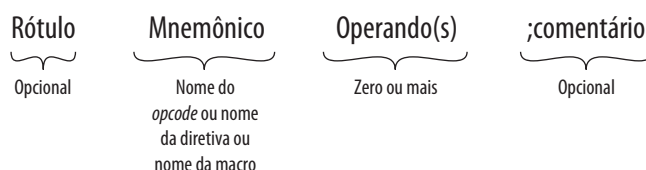
Motivos para usar um rótulo incluem:

1. Um rótulo torna uma posição do programa mais fácil de localizar e lembrar.
2. O rótulo pode ser facilmente movido para corrigir um programa. O montador automaticamente mudará o endereço em todas as instruções que usam o rótulo quando o programa for remontado.
3. O programador não precisa calcular endereços de memória relativos ou absolutos, mas apenas usa rótulos conforme necessário.

MNEMÔNICO O mnemônico é o nome da operação ou função da sentença da linguagem de montagem. Conforme discutido a seguir, uma sentença pode corresponder a uma instrução de máquina, uma diretiva do montador ou uma macro. No caso de uma instrução de máquina, um mnemônico é o nome simbólico associado com um determinado *opcode*.

A Tabela 10.8 mostra o mnemônico, ou nome da instrução, de muitas instruções x86. O Apêndice A de Carter (2006^b) mostra as instruções x86 junto com os operandos de cada uma e o efeito da instrução nos códigos condicionais. O Apêndice B do manual de NASM fornece uma descrição mais detalhada de cada instrução x86. Ambos os documentos estão disponíveis no site deste livro.

Figura B.1 Estrutura da sentença da linguagem de montagem



OPERANDO(S) Uma sentença da linguagem de montagem inclui zero ou mais operandos. Cada operando identifica um valor imediato, um registrador ou uma posição de memória. Normalmente, a linguagem de montagem fornece convenções para distinguir entre os três tipos de referências de operandos, assim como convenções para indicar o modo de endereçamento.

Para arquitetura x86, um comando da linguagem de montagem pode referir-se ao operando registrador pelo nome. A Figura B.2 ilustra os registradores de propósito geral do x86 e do com o seu nome simbólico e sua codificação em bits. O montador vai traduzir o nome simbólico em um identificador binário do registrador.

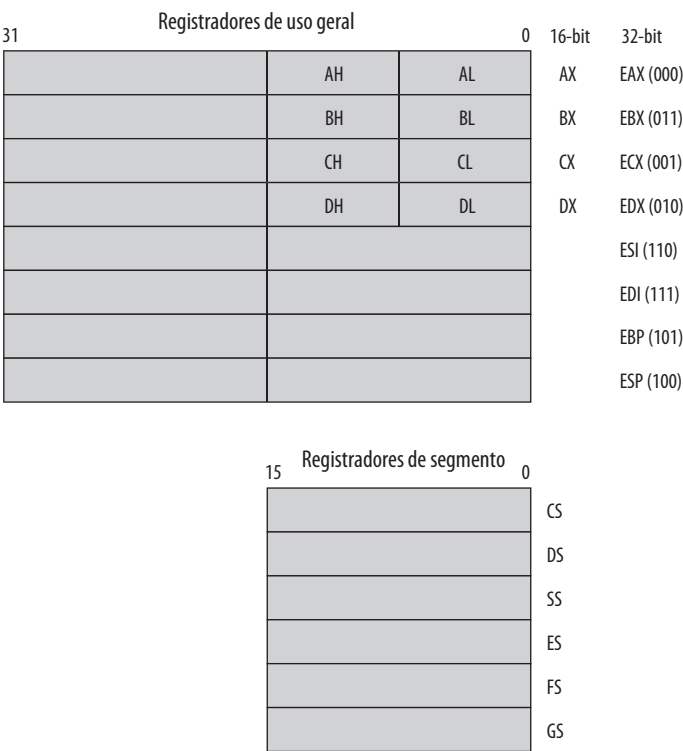
Conforme discutido na Seção 11.2, a arquitetura x86 possui um conjunto rico de modos de endereçamento, cada um tendo que ser expresso simbolicamente na linguagem de montagem. Aqui citamos alguns dos exemplos mais comuns. Para **endereçamento de registrador**, o nome do registrador é usado na instrução. Por exemplo, **MOV ECX, EBX** copia o conteúdo do registrador EBX para o registrador ECX. O endereçamento imediato indica que o valor é codificado dentro da instrução. Por exemplo, **MOV EAX, 100H** copia o valor hexadecimal 100 no registrador EAX. O valor imediato pode ser expresso como um número binário com sufixo B ou um número decimal sem sufixo. Assim, as sentenças equivalentes à anterior são **MOV EAX, 100000000B** e **MOV EAX, 256**. O **endereçamento direto** refere-se a uma posição de memória e é expresso como um deslocamento a partir do registrador de segmento DS. Isto é explicado melhor com um exemplo. Suponha que o registrador de segmentos de dados de 16 bits DS contenha o valor 1000H. Então, ocorre a seguinte sequência:

```
MOV AX, 1234H
MOV [3518H], AX
```

Primeiro, o registrador AX de 16 bits é inicializado com 1234H. Depois, na linha dois, o conteúdo de AX é movido para endereço lógico DS:3518H. Este endereço é formado pelo deslocamento do conteúdo de DS de 4 bits para esquerda e adicionando 3518H para formar o endereço lógico 13518H de 32 bits.

COMENTÁRIO Todas as linguagens de montagem permitem colocar comentários dentro do programa. Um comentário pode ocorrer do lado direito de um comando em linguagem ou pode ocupar uma linha de texto inteira.

Figura B.2 Registradores de execução de programa do Intel x86



Nos dois casos, o comentário começa com um caractere especial que sinaliza para o montador que o restante da linha é um comentário e deve ser ignorado pelo montador. Normalmente, as linguagens de montagem para arquitetura x86 usam ponto e vírgula (;) como caractere especial.



Tipos de sentenças da linguagem de montagem

As sentenças da linguagem de montagem podem ser de um dos quatro tipos: instrução, diretiva, definição de macro e comentário. Uma sentença comentário é simplesmente uma sentença que consiste inteiramente de um comentário. Outros tipos são descritos brevemente nesta seção.

INSTRUÇÕES Muitas das sentenças em um programa da linguagem de montagem são representações simbólicas de instruções de linguagem de máquina. Quase que invariavelmente, há um relacionamento de um para um entre uma instrução da linguagem de montagem e uma instrução de máquina. O montador resolve quaisquer referências simbólicas e traduz instruções da linguagem de montagem em cadeias binárias que representam uma instrução de máquina.

DIRETIVAS Diretivas, também chamadas de **pseudoinstruções**, são sentenças da linguagem de montagem que não são diretamente traduzidas para instruções da linguagem de máquina. Em vez disso, as diretivas são instruções para o montador executar ações específicas durante o processo de montagem. Exemplos incluem o seguinte:

- Definir constantes.
- Designar áreas da memória para armazenar dados.
- Inicializar áreas da memória.
- Colocar tabelas ou outros dados fixos na memória.
- Permitir referências para outros programas.

A Tabela B.2 lista algumas diretivas do NASM. Como um exemplo, considere a seguinte sequência de sentenças:

Tabela B.2 Algumas diretivas da linguagem de montagem do NASM

(a) Letras para diretivas RESx e Dx

Unidade	Letra
Byte	B
Palavra (2 bytes)	W
Palavra dupla (4 bytes)	D
Palavra quádrupla (8 bytes)	Q
Dez bytes	T

(b) Diretivas

Nome	Descrição	Exemplo
DB, DW, DD, DQ, DT	Inicializa posições	L6 DD 1A92H ; palavra dupla em L6 inicializada com 1A92H
RESB, RESW, RESD, RESQ, REST	Reserva posições não inicializadas	BUFFER RESB 64 ; reserva 64 bytes começando em BUFFER
INCBIN	Inclui arquivo binário na saída	INCBIN "file.dat" ; inclui este arquivo
EQU	Define um símbolo para um dado valor constante	MSGLEN EQU 25 ; constante MSGLEN equivale ao decimal 25
TIMES	Repete instrução várias vezes	ZEROBUF TIMES 64 DB 0 ; inicializa buffer de 64 bytes todo para zeros

```

L2 DB    "A"        ; byte inicializado para código ASCII de A (65)
MOV  AL, [L1]        ; copiar byte que está em L1 para AL
MOV  EAX, L1         ; armazenar endereço do byte que está em L1 em EAX
MOV  [L1], AH        ; copiar conteúdo de AH dentro do byte que está em L1

```

Se um rótulo direto é usado, ele é interpretado como endereço (ou offset) de dados. Se o rótulo é colocado dentro de colchetes, ele interpretado como dado no endereço.

DEFINIÇÕES DE MACRO Uma definição de macro é semelhante a uma sub-rotina de várias formas. Uma sub-rotina é uma seção do programa que é escrita uma vez e pode ser usada várias vezes, chamando a sub-rotina a partir de qualquer ponto do programa. Quando o programa é compilado ou montado, a sub-rotina é carregada apenas uma vez. Uma chamada da sub-rotina transfere controle para a sub-rotina e uma instrução da sub-rotina retorna o controle para o ponto de chamada. De forma semelhante, uma definição de macro é uma seção de código que o programador escreve uma vez e pode depois usar várias vezes. A principal diferença é que quando o montador encontra uma chamada de macro, ele substitui a chamada de macro pela macro em si. O processo é chamado de **expansão de macro**. Então, se uma macro é definida em um programa da linguagem de montagem e é chamada 10 vezes, então 10 instâncias da macro irão aparecer no código montado. Basicamente, sub-rotinas são tratadas pelo hardware em tempo de execução, enquanto que as macros são tratadas pelo montador em tempo de montagem. Macros fornecem a mesma vantagem que as sub-rotinas em termos de programação modular, mas sem a sobrecarga em tempo de execução de chamada e retorno de uma sub-rotina. O custo disso é que a abordagem de macro usa mais espaço no código objeto.

No NASM e em muitos outros montadores, uma distinção é feita entre uma macro de uma única linha e uma macro de várias linhas. No NASM, macros de linha única são definidas usando diretiva `%DEFINE`. Aqui está um exemplo em que várias macros de única linha são expandidas. Primeiro definimos duas macros:

```

%DEFINE  B(X) = 2*X
%DEFINE  A(X) = 1 + B(X)

```

Em algum ponto do programa da linguagem de montagem, aparecem as seguintes sentenças:

```
MOV AX, A(8)
```

O montador expande esta sentença para:

```
MOV AX, 1+2*8
```

o que é montado para uma instrução de máquina para mover o valor imediato 17 para o registrador AX.

Macros de várias linhas são definidas usando o mnemônico `%MACRO`. Aqui está o exemplo de uma definição de macro de várias linhas:

```

%MACRO PROLOGUE 1
    PUSH EBP          ; coloca conteúdo de EBP na pilha
                      ; apontada por ESP e
                      ; decrementa conteúdo de ESP em 4
    MOV EBP, ESP       ; copia conteúdo de ESP para EBP
    SUB ESP, %1        ; subtrai o valor do primeiro parâmetro de ESP
%ENOMACRO

```

O número 1 depois do nome da macro na linha `%MACRO` define o número de parâmetros que a macro espera receber. O uso de `%1` dentro da definição da macro refere-se ao primeiro parâmetro da chamada da macro.

A chamada da macro

```
MYFUNC: PROLOGUE 12
```

expande para as seguintes linhas de código:

```
MYFUNC:  PUSH    EBP
         MOV     EBP,    ESP
         SUB     ESP,    12
```



Exemplo: programa do maior divisor comum

Como um exemplo de uso da linguagem de montagem, analisamos um programa para calcular o maior divisor comum de dois números inteiros. Definimos o maior divisor comum dos inteiros *a* e *b* da seguinte forma:

$\text{gcd}(a, b) = \max[k, \text{tal que } k \text{ é divisor de } a \text{ e } k \text{ divisor de } b]$

onde dizemos que *k* é divisor de *a* se não houver resto de divisão. O algoritmo de Euclides para o maior divisor comum é baseado no seguinte teorema. Para quaisquer inteiros positivos *a* e *b*,

$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$

Aqui está um programa na linguagem C que implementa algoritmo de Euclides:

```
unsigned int gcd (unsigned int a, unsigned int b)
{
    if (a == 0 && b == 0)
        b = 1;
    else if (b == 0)
        b = a;
    else if (a != 0)
        while (a != b)
            if (a < b)
                b -= a;
            else
                a -= b;
    return b;
}
```

A Figura B.3 mostra duas versões na linguagem de montagem do programa anterior. O programa à esquerda foi feito por um compilador C; o programa à direita foi programado à mão. O último usa uma série de truques de programação para produzir uma implementação mais compacta e eficiente.



B.2 Montadores

O montador é um software utilitário que recebe como entrada um programa em linguagem de máquina (*assembly*) e produz o código objeto como saída. O código objeto é um arquivo binário. O montador enxerga esse arquivo como um bloco de memória iniciando na posição relativa 0.

Existem duas abordagens gerais para os montadores: montador de dois passos e montador de um passo.



Montador de dois passos

Analisamos primeiro o montador de dois passos, que é mais comum e um pouco mais fácil de entender. O montador faz duas passagens pelo código fonte (Figura B.4):

PRIMEIRO PASSO No primeiro passo, o montador se preocupa apenas com definições dos rótulos. O primeiro passo é usado para construir uma *tabela de símbolos* que contém uma lista de todos os rótulos e seus valores de *contador de posição* (LC, do inglês *location counter*). O primeiro byte do código objeto terá o valor 0 como LC.

Figura B.3 Programas de montagem para o maior divisor comum

<pre> gcd: mov ebx, eax mov eax, edx test ebx, ebx jne L1 test edx, edx jne L1 mov eax, 1 ret L1: test eax, eax jne L2 mov eax, ebx ret L2: test ebx, ebx je L5 L3: cmp ebx, eax je L5 jae L4 sub eax, ebx jmp L3 L4: sub ebx, eax jmp L3 L5: ret </pre>	<pre> gcd: neg eax je L3 L1: neg eax xchg eax, edx L2: sub eax, edx jg L2 jne L1 L3: add eax, edx jne L4 inc eax L4: ret </pre>
(a) Programa compilado	(b) Escrito diretamente na linguagem de montagem

O primeiro passo examina cada sentença em linguagem de montagem. Embora o montador ainda não esteja pronto para traduzir instruções, ele tem que examinar cada instrução suficientemente para determinar o tamanho da instrução de máquina correspondente e, portanto, em quanto incrementar o LC. Isso pode requerer não apenas a análise do *opcode*, mas também a análise dos operandos e dos modos de endereçamento.

Diretivas como DQ e REST (veja Tabela B.2) fazem o contador de posição ser ajustado de acordo com a quantidade de armazenamento necessário.

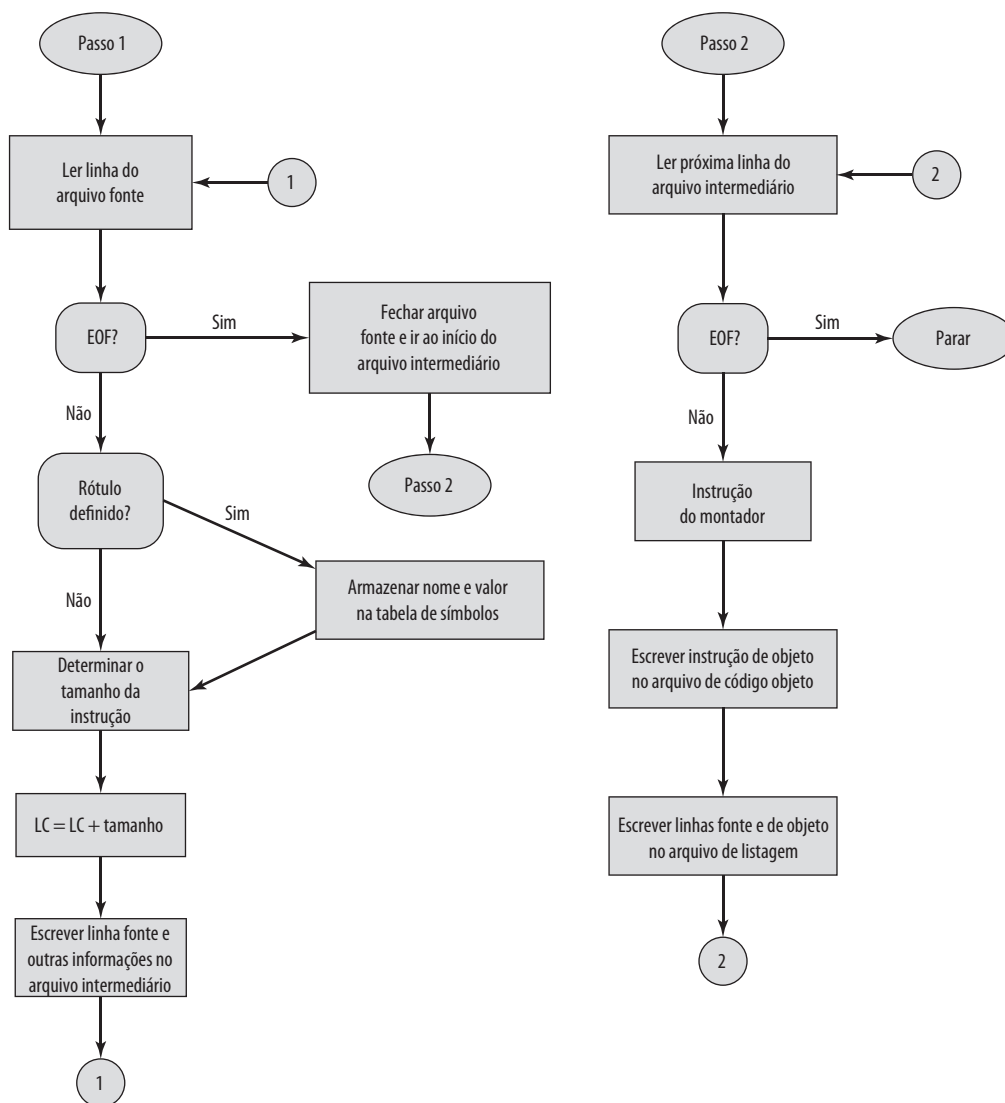
Quando o montador encontra uma sentença com um rótulo, ele coloca o rótulo na tabela de símbolos, junto com o valor corrente de LC. O montador continua até que tenha lido todas as sentenças da linguagem de montagem.

SEGUNDO PASSO O segundo passo lê o programa novamente desde o começo. Cada instrução é traduzida no código binário de máquina apropriado. A tradução envolve as seguintes operações:

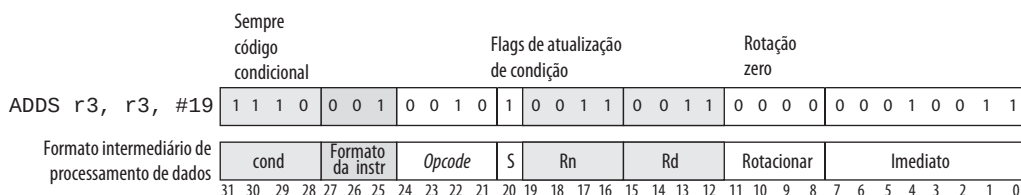
1. Traduzir mnemônico em *opcode* binário.
2. Usar o *opcode* para determinar o formato da instrução, a posição e o tamanho de vários campos na instrução.
3. Traduzir o nome de cada operando para o registrador ou código de memória apropriado.
4. Traduzir cada valor imediato em uma cadeia binária.
5. Traduzir quaisquer referências a rótulos em valores de LC apropriados usando a tabela de símbolos.
6. Definir quaisquer outros bits necessários dentro da instrução, incluindo indicadores do modo de endereçamento, bits de códigos condicionais e assim por diante.

Um exemplo simples, usando linguagem de montagem ARM, é mostrado na Figura B.5. A instrução da linguagem de montagem ARM ADDS, r3, r9, #19 é traduzida para instrução binária de máquina 1110 0010 0101 0011 0011 0000 0001 0011.

PASSO ZERO A maioria das linguagens de montagem inclui a capacidade de definir macros. Quando elas estão presentes, há um passo adicional que o montador deve fazer antes do primeiro passo. Normalmente, a linguagem de montagem requer que todas as definições de macro apareçam no início do programa.

Figura B.4 Fluxograma do montador de dois passos

O montador inicia este “passo zero” lendo todas as definições de macro. Uma vez reconhecidas todas as macros, o montador vai pelo código fonte e as expande, com seus parâmetros associados, sempre que uma chamada de macro é encontrada. O passo de processamento de macros gera um nova versão do código fonte com todas as expansões das macros no devido lugar e todas as definições de macro removidas.

Figura B.5 Traduzindo uma instrução em linguagem de montagem do ARM para uma instrução binária de máquina



Montador de um passo

É possível implementar um montador que faz uma única passagem pelo código fonte (sem contar o passo para processar macros). A principal dificuldade em tentar montar um programa em uma única passagem envolve referências futuras a rótulos. Os operandos das instruções podem ser símbolos que ainda não foram definidos no programa fonte. Portanto, o montador não sabe qual endereço relativo inserir na instrução traduzida.

Basicamente, o processo para resolver referências futuras funciona da seguinte forma. Quando o montador encontra um operando da instrução que é um símbolo ainda não definido, ele:

1. Deixa o campo do operando da instrução vazio (tudo zero) na instrução binária montada.
2. Insere o símbolo usado como um operando na tabela de símbolos. A entrada da tabela é marcada para indicar que o símbolo não está definido.
3. Adiciona o endereço do campo de operando da instrução que se refere ao símbolo indefinido a uma lista de referências futuras associadas com a entrada na tabela de símbolos.

Quando a definição de símbolo é encontrada de tal forma que o valor LC possa ser associado a ele, o montador insere o valor LC na entrada adequada dentro da tabela de símbolos. Se há uma lista de referência futura associada com o símbolo, então o montador insere o endereço apropriado em qualquer instrução gerada previamente que esteja na lista de referência futura.



Exemplo: programa de números primos

Analisamos agora um exemplo que inclui diretivas. Este exemplo analisa um programa que acha números primos. Lembre que os números primos são divisíveis apenas por 1 e por si mesmos. Não há uma fórmula para fazer isso. O método básico usado por este programa é localizar fatores de todos os números ímpares abaixo de um dado limite. Se nenhum fator puder ser encontrado para um número ímpar, então ele é um número primo. A Figura B.6 mostra o algoritmo básico escrito em C. A Figura B.7 mostra o mesmo algoritmo escrito na linguagem de montagem NASM.

Figura B.6 Programa C para verificar os números primos

```
unsigned guess;           /* suposição atual para primo */
unsigned factor;          /* fator possível para suposição */
unsigned limit;           /* encontrar primos até este valor */

printf("Find primes up to : ");
scanf("%u", &limit);
printf("2\n");            /* tratar dois primeiros primos */
printf("3\n");            /* caso especial */
guess = 5;                /* suposição inicial */
while (guess <= limit) {   /* procurar por um fator da suposição */
    factor = 3;
    while (factor * factor < guess && guess % factor != 0)
        factor += 2;
    if (guess % factor != 0)
        printf("%d\n", guess);
    guess += 2;            /* analisar apenas números ímpares */
}
```


Figura B.7 Programa em linguagem de montagem para verificar números primos

```

#include "asm_io.inc"
segment .data
Message db "Find primes up to: ", 0

segment .bss
Limit resd 1                ; encontrar primos até este limite
Guess resd 1                ; suposição atual para primo

segment .text
global _asm_main
_asm_main:
    enter 0,0                ; rotina de setup
    pusha

    mov eax, Message
    call print_string
    call read_int             ; scanf("%u", & limit);
    mov [Limit], eax
    mov eax, 2                ; printf("2\n");
    call print_int
    call print_nl
    mov eax, 3                ; printf("3\n");
    call print_int
    call print_nl

    mov dword [Guess], 5     ; Guess = 5;
while_limit:                 ; while (Guess <= Limit)
    mov eax, [Guess]
    cmp eax, [Limit]
    jnbe end_while_limit     ; usar jnbe porque os números não têm sinal

    mov ebx, 3                ; ebx is factor = 3;
while_factor:
    mov eax, ebx
    mul eax                   ; edx:eax = eax*eax
    jo end_while_factor      ; se a resposta não couber apenas em eax
    cmp eax, [Guess]
    jnb end_while_factor     ; if !(factor*factor < guess)
    mov eax, [Guess]
    mov edx, 0
    div ebx                   ; edx = edx:eax% ebx
    cmp edx, 0
    je end_while_factor      ; if !(guess% factor != 0)

    add ebx, 2; factor += 2;
    jmp while_factor
end_while_factor:
    je end_if                 ; if !(guess% factor != 0)
    mov eax, [Guess]
    call print_int
    call print_nl
end_if:
    add dword [Guess], 2      ; guess += 2
    jmp while_limit
end_while_limit:

    popa
    mov eax, 0                ; volta para C
    leave
    ret

```

B.3 Carregamento e ligação

O primeiro passo na criação de um processo ativo é carregar o programa na memória principal e criar uma imagem de processo (Figura B.8). A Figura B.9 ilustra um cenário típico para a maioria dos sistemas. A aplicação consiste de uma série de módulos compilados ou montados na forma de código objeto. Estes são vinculados para resolver quaisquer referências entre módulos. Ao mesmo tempo, referências às rotinas de biblioteca são resolvidas. As rotinas de biblioteca, por sua vez, podem ser incorporadas no programa ou referenciadas como código compartilhado que deve ser fornecido pelo sistema operacional em tempo de execução. Nesta seção resumimos os principais recursos dos ligadores e carregadores (*linkers* e *loaders*). Primeiramente discutimos o conceito de relocação. Depois, para esclarecer a apresentação, descrevemos a tarefa de carregamento quando um único módulo de programa está envolvido; nenhuma ligação é necessária. Depois podemos analisar vinculação e carregamento de funções como um todo.

Realocação

Em um sistema multiprogramado, a memória principal disponível geralmente é compartilhada entre uma série de processos. Normalmente, não é possível que o programador saiba de antemão quais outros processos estarão residentes na memória principal no momento da execução do seu programa. Além disso, gostaríamos de poder transferir processos ativos para dentro e para fora da memória principal para maximizar a utilização do processador fornecendo um conjunto grande de processos prontos para executar. Uma vez o processo sendo transferido para o disco, seria muito limitante declarar que, quando fosse transferido de volta, deveria ser colocado na mesma região da memória principal como antes. Em vez disso, podemos ter que **realocar** o processo para uma área diferente da memória.

Desta forma, não conseguimos saber de antemão onde o programa será colocado e temos que permitir que o programa possa ser movido dentro da memória principal por causa das transferências. Estes fatos trazem à tona algumas preocupações técnicas relacionadas com endereçamento, conforme ilustrado na Figura B.10. Ela ilustra uma imagem de processo. Para simplificar, vamos supor que a imagem de processo esteja ocupando uma região contínua da memória principal. Claramente, o sistema operacional terá que saber a posição da informação de

Figura B.8 Função de carregamento

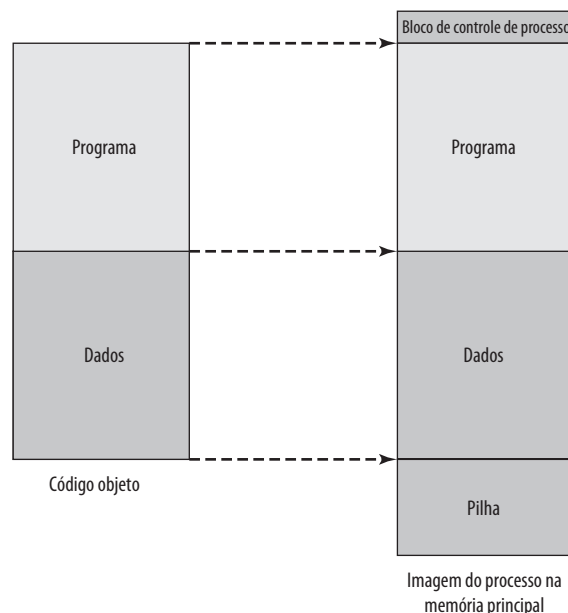
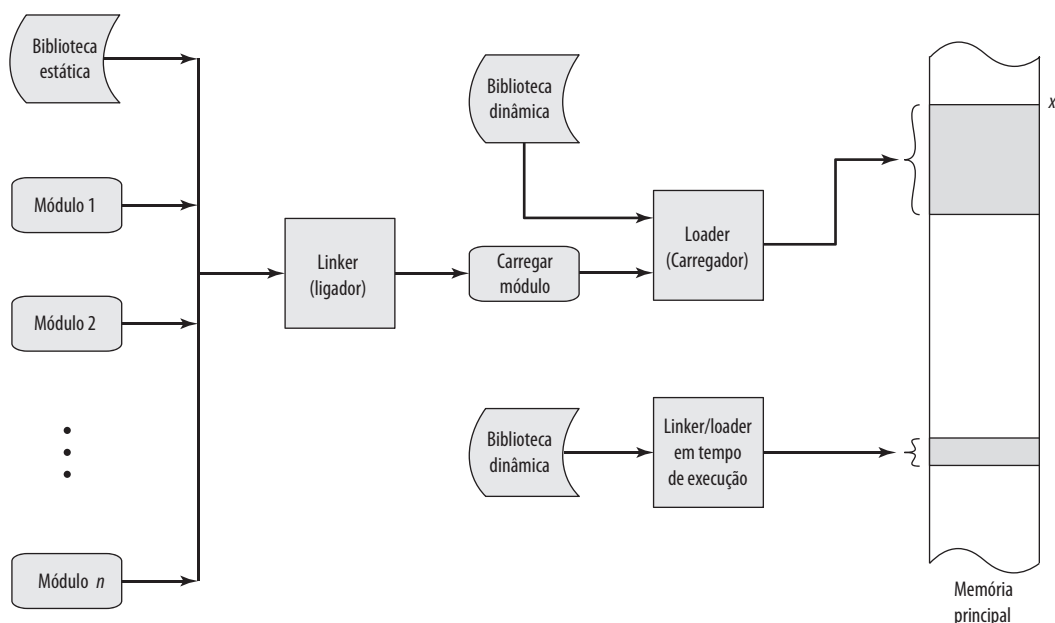
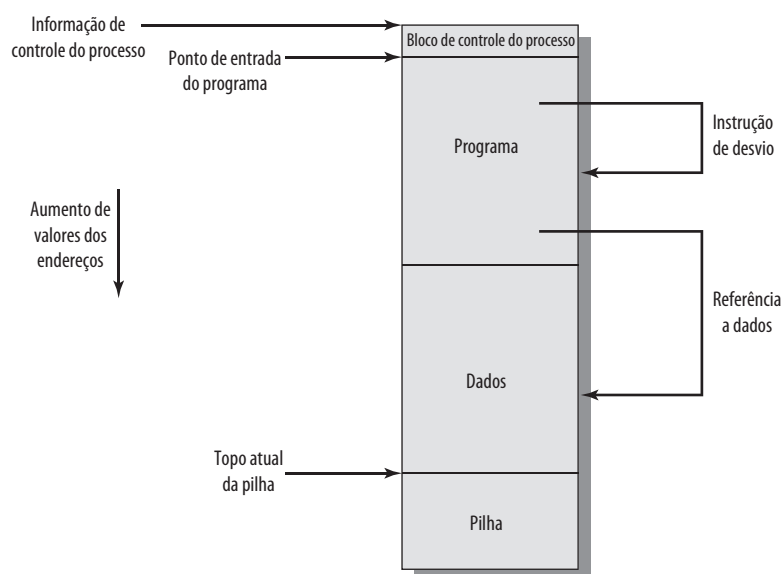


Figura B.9 Um cenário de ligação e carregamento

controle do processo e da pilha de execução, assim como o ponto de entrada para começar a execução do programa para este processo. Como o sistema operacional está gerenciando a memória e é responsável por trazer esse processo para a memória principal, é fácil chegar até esses endereços. Além disso, o processador precisa lidar com referências de memória dentro do programa. As instruções de desvio contêm um endereço para referenciar a instrução a ser executada a seguir. As instruções de referência a dados contêm o endereço do byte ou da palavra

Figura B.10 Requisitos de endereçamento para um processo

de dados referenciados. De alguma forma, o hardware do processador e o software do sistema operacional devem ser capazes de traduzir as referências de memória encontradas no código do programa para endereços físicos de memória reais, refletindo a posição atual do programa na memória principal.



Carregamento

Na Figura B.9, o carregador (loader) coloca o módulo na memória principal iniciando em posição *x*. Ao carregar o programa, o requisito de endereçamento ilustrado na Figura B.10 deve ser satisfeito. Em geral, três abordagens podem ser tomadas:

- Carregamento absoluto.
- Carregamento realocável.
- Carregamento dinâmico em tempo de execução.

CARREGAMENTO ABSOLUTO Um carregador absoluto requer que um dado módulo sempre seja carregado na mesma posição da memória principal. Assim, no módulo carregável apresentado ao carregador, todas as referências de endereço devem ser para endereços específicos, ou absolutos, da memória principal. Por exemplo, se *x* na Figura B.9 for posição 1024, então a primeira palavra em um módulo a ser carregado destinado para essa região da memória tem endereço 1024.

A atribuição de valores de endereço específicos para referências de memória dentro de um programa pode ser feita pelo programador ou em tempo de compilação ou execução (Tabela B.3a). Existem várias desvantagens para abordagem anterior. Em primeiro lugar, todo programador deveria saber a estratégia de atribuição pretendida para colocar módulos na memória principal. Em segundo lugar, se quaisquer modificações forem feitas para o programa que envolva inclusão ou exclusão no corpo do módulo, então todos os endereços terão que ser alterados. Consequentemente, é preferível permitir que as referências de memória dentro dos programas sejam expressas simbolicamente e depois resolver essas referências simbólicas em tempo de compilação ou montagem. Isso é

Tabela B.3 Ligação de endereços

(a) Loader (carregador)	
Tempo da ligação	Função
Tempo de programação	Todos os endereços físicos reais são especificados diretamente pelo programador dentro do próprio programa.
Tempo de compilação ou montagem	O programa contém referências de endereços simbólicas e estas são convertidas para endereços físicos reais pelo compilador ou montador.
Tempo de carregamento	O compilador ou o montador produz endereços relativos, que são traduzidos, pelo carregador para endereços absolutos em tempo de carregamento do programa.
Tempo de execução	O programa carregado retém endereços relativos. Estes são convertidos dinamicamente para endereços absolutos pelo hardware do processador.

(b) Linker (ligador)	
Tempo de vinculação	Função
Tempo de programação	Nenhuma referência a programas externos ou dados é permitida. O programador deve colocar no programa o código fonte para todos os subprogramas que são referenciados.
Tempo de compilação ou montagem	O montador deve obter o código fonte de cada subrotina que é referenciada e montá-la como uma unidade.
Criação do módulo a ser carregado	Todos os módulos objetos foram montados usando endereços relativos. Esses módulos são vinculados entre si e todas as referências são refeitas com relação à origem do último módulo.
Tempo de carregamento	Referências externas não são resolvidas até que o módulo esteja para ser carregado na memória principal. Nesse momento, os módulos de vinculação dinâmica referenciados são acrescidos ao módulo a ser carregado e o pacote inteiro é carregado na memória principal ou virtual.
Tempo de execução	Referências externas não são resolvidas até que a chamada externa seja executada pelo processador. Nesse momento, o processo é interrompido e o módulo desejado é vinculado ao programa que fez a chamada.

ilustrado na Figura B.11. Toda referência para uma instrução ou item de dados é representada inicialmente por um símbolo. Ao preparar o módulo para entrar em um carregador absoluto, o montador ou o compilador converterá todas essas referências para endereços específicos (neste exemplo, para um módulo para ser carregado iniciando na posição 1024), conforme mostrado na Figura B.11b.

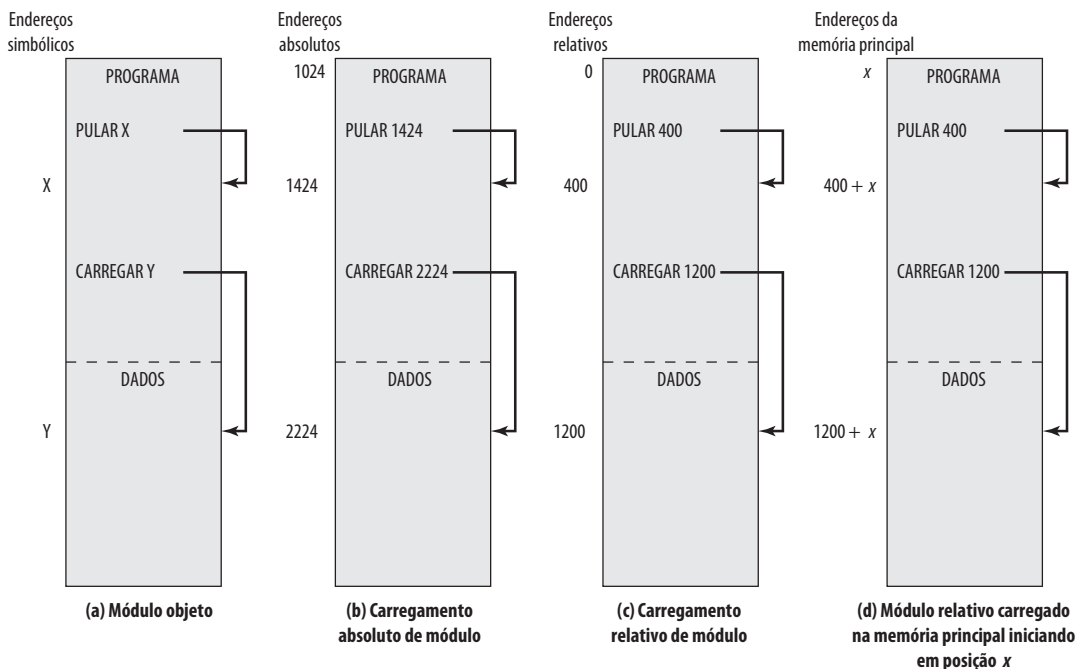
CARREGAMENTO REALOCÁVEL A desvantagem de ligar referências de memória a endereços específicos antes do carregamento é que o módulo resultante a ser carregado pode ser colocado em apenas uma região da memória principal. No entanto, quando muitos programas compartilham a memória principal, pode não ser desejável decidir antecipadamente em qual região da memória um determinado módulo deve ser carregado. É melhor tomar essa decisão em tempo de carregamento. Precisamos, assim, de um módulo que pode ser alocado em qualquer lugar da memória principal.

Para satisfazer esse novo requisito, o montador ou compilador produz não os endereços reais da memória principal (endereços absolutos), e sim os endereços relativos a algum ponto conhecido, como o início do programa. Esta técnica é ilustrada na Figura B.11c. Ao início do módulo carregável é atribuído o endereço relativo 0, e todas as outras referências de memória dentro do módulo são expressas com relação ao início do módulo.

Com todas as referências de memória expressas no formato relativo, torna-se fácil para o carregador colocar o módulo na posição desejada. Se o módulo for carregado começando na posição x , então o carregador deve simplesmente adicionar x para cada referência de memória à medida que carrega o módulo na memória. Para ajudar nesta tarefa, o módulo carregável deve incluir informação que diz ao carregador onde estão as referências de memória e como devem ser interpretadas (normalmente serão relativas à origem do programa, mas também possivelmente relativas a algum outro ponto no programa, como por exemplo a posição atual). Este conjunto de informações é preparado pelo compilador ou montador e normalmente é chamado de dicionário de realocação.

CARREGAMENTO DINÂMICO EM TEMPO DE EXECUÇÃO Carregadores realocáveis são comuns e fornecem benefícios óbvios se comparados aos carregadores absolutos. No entanto, em um ambiente de multiprogramação, até naquele que não depende da memória virtual, o esquema de carregamento realocável é inadequado. Já mencionamos a necessidade de transferir imagens de processo para dentro e para fora da memória principal para maximizar a utilização do processador. Para maximizar a utilização da memória principal, gostaríamos de ser capazes de

Figura B.11 Carregamento de módulos absoluto e realocável



transferir a imagem do processo de volta para posições diferentes em tempos diferentes. Assim, uma vez carregado, um programa pode ser transferido para o disco e depois ser transferido de volta para uma posição diferente. Isto seria impossível se as referências de memória tivessem sido ligadas a endereços absolutos no momento do carregamento inicial.

A alternativa é adiar o cálculo de um endereço absoluto até que seja realmente necessário em tempo de execução. Para este propósito, o módulo é carregado na memória principal com todas as referências de memória na forma relativa (Figura B.11c). O endereço absoluto não é calculado até que uma instrução seja executada. Para garantir que essa funcionalidade não prejudique o desempenho, ela deve ser executada pelo hardware do processador especial em vez do software. Este hardware é descrito no Capítulo 8.

O cálculo dinâmico de endereços fornece flexibilidade completa. Um programa pode ser carregado em qualquer região da memória principal. Subsequentemente, a execução do programa pode ser interrompida e o programa pode ser transferido para fora da memória principal, para depois ser transferido de volta para uma posição diferente.



Ligação

A função de um ligador (*linker*) é receber como entrada uma coleção de módulos de objeto e produzir um módulo carregável, consistindo de um conjunto integrado de módulos de programa e dados para ser passado para o carregador. Em cada módulo objeto pode haver referências de endereços para posições em outros módulos. Cada referência dessas apenas pode ser expressa simbolicamente em um módulo objeto não vinculado. O *linker* cria um único módulo carregável que é uma junção contínua de todos os módulos objeto. Por exemplo, o módulo A na Figura B.12a contém uma chamada de um procedimento do módulo B. Quando esses módulos são combinados no módulo carregável, esta referência simbólica para o módulo B é alterada para uma referência específica na posição do ponto de entrada de B dentro do módulo carregável.

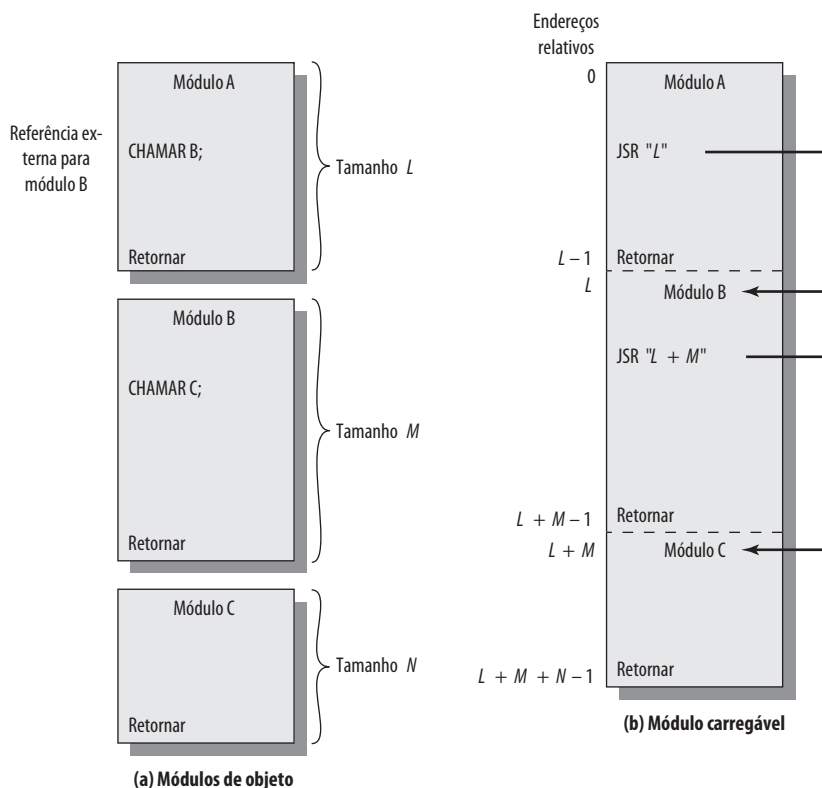
EDITOR DE LIGAÇÃO (*linkage editor*) A natureza desta ligação de endereços vai depender do tipo de módulo carregável a ser criado e de quando a ligação ocorre (Tabela B.3b). Se, como acontece normalmente, um módulo carregável realocável for desejado, então a ligação é normalmente feita da seguinte maneira. Cada módulo objeto compilado ou montado é criado com referências relativas ao início do módulo objeto. Todos esses módulos são juntados em um único módulo carregável realocável com todas as referências relativas à origem do módulo carregável. Este módulo pode ser usado com entrada para carregamento realocável ou carregamento dinâmico em tempo de execução.

Um *linker* que produz um módulo carregável realocável frequentemente é referido como um editor de ligação (*linkage editor*). A Figura B.12 ilustra a função do editor de ligação.

LIGAÇÃO DINÂMICA Como acontece com carregamento, é possível adiar algumas funções de ligação. O termo *ligação dinâmica* é usado para se referir à prática de adiar a ligação de alguns módulos externos até depois de o módulo carregável ter sido criado. Desta forma, o módulo carregável contém referências não resolvidas para outros programas. Essas referências podem ser resolvidas em tempo de carregamento ou em tempo de execução.

Para **ligação dinâmica em tempo de carregamento** (envolvendo biblioteca dinâmica superior na Figura B.9), ocorrem os seguintes passos. O módulo carregável (módulo da aplicação) a ser carregado é lido para memória. Qualquer referência para um módulo externo (módulo alvo) faz com que o carregador localize o módulo alvo, carregue-o e altere a referência para um endereço na memória relativa ao início do módulo da aplicação. Existem várias vantagens para esta abordagem quando comparada ao que podemos chamar de ligação estática:

- Torna-se mais fácil incorporar versões alteradas ou atualizadas do módulo alvo, o que pode ser um utilitário do sistema operacional ou alguma outra rotina de propósito geral. Com ligação estática, uma mudança em um módulo de suporte como esse iria requerer a religação do módulo da aplicação inteiro. Não apenas isso é ineficiente, mas pode ser impossível em algumas circunstâncias. Por exemplo, no campo de computadores pessoais, a maioria dos softwares comerciais é distribuída na forma de módulo carregável; versões fonte e objeto não são fornecidas.
- Ter código alvo em um arquivo de ligação dinâmico cria o caminho para compartilhamento automático de código. O sistema operacional pode reconhecer que mais do que uma aplicação está usando o mesmo código alvo porque ela carregou e ligou esse código. Ele pode usar essa informação para carregar uma única cópia do código alvo e ligá-lo para ambas as aplicações, em vez de ter que carregar uma cópia para cada aplicação.

Figura B.12 A função de vinculação

- Torna-se mais fácil para desenvolvedores independentes de software estender as funcionalidades de um sistema amplamente usado como Linux. Um desenvolvedor pode aparecer com uma nova função que pode ser útil para uma variedade de aplicações e distribuí-la como um módulo de ligação dinâmica.

Com a **ligação dinâmica em tempo de execução** (envolvendo biblioteca dinâmica inferior na Figura B.9), algumas ligações são adiadas até o momento da execução. Referências externas para módulos alvos permanecem no programa carregado. Quando uma chamada é feita para módulo ausente, o sistema operacional localiza o módulo, carrega-o e liga-o com o módulo que fez a chamada. Tais módulos normalmente são compartilhados. No ambiente Windows, esses módulos são chamados de bibliotecas de ligação dinâmica (DLL, do inglês *dynamic-link libraries*). Assim, se um processo já está fazendo uso de um módulo compartilhado ligado dinamicamente, então o módulo está na memória principal e um novo processo pode simplesmente ligar o módulo já carregado.

O uso de DLL pode levar a um problema normalmente chamado de **inferno de DLL**. Isso ocorre se dois ou mais processos estão compartilhando um módulo DLL, porém esperam versões diferentes do módulo. Por exemplo, uma aplicação ou uma função do sistema pode ser reinstalada e trazer consigo uma versão mais antiga de um arquivo DLL.

Vimos que carregamento dinâmico permite que um módulo carregável inteiro seja movido; no entanto, a estrutura do módulo é estática, permanecendo inalterada durante a execução do processo e de uma execução para outra. No entanto, em alguns casos, não é possível determinar antes da execução quais módulos de objeto serão necessários. Esta situação é comum em aplicações com processamento transacional, como um sistema de reservas aéreas ou uma aplicação bancária. A natureza da transação dita quais módulos de programa são necessários e eles são carregados conforme necessário e ligados ao programa principal. A vantagem do uso de tal ligado dinâmico é que não é necessário alocar memória para unidades do programa a não ser que essas unidades sejam referenciadas. Essa capacidade é usada para suportar sistemas de segmentação.

Um aprimoramento adicional é possível: uma aplicação não precisa saber os nomes de todos os módulos ou pontos de entrada que podem ser chamados. Por exemplo, um programa gráfico pode ser escrito para trabalhar com variedades de plotters, dos quais cada um é controlado por um driver diferente. A aplicação pode aprender o

nome do ploter que está instalado atualmente no sistema a partir de outro processo ou procurando por ele em um arquivo de configuração. Isso permite ao usuário da aplicação instalar um novo ploter que não existia no tempo em que a aplicação foi escrita.



B.4 Leitura recomendada e sites Web

Salomon (1993^c) cobre o projeto e a implementação de montadores e carregadores.

Os assuntos sobre ligação e carregamento são cobertos em muitos livros sobre desenvolvimento de programas, arquitetura de computação e sistemas operacionais. Uma obra particularmente detalhada é Beck (1997^d). Clarke e Merusi (1998^e) também contém uma boa discussão. Uma discussão prática aprofundada deste assunto, com vários exemplos de SO, é Levine (2000^f).

Bartlett (2003^g) é uma obra excelente para aprender linguagem de montagem para processadores x86, adequada para autoestudo. Carter (2006^b) cobre linguagem de montagem para máquinas x86. Para o programador x86 sério, Fog (2008^a) é muito útil. Knaggs e Welsh (2004^h) é uma obra aprofundada sobre linguagem de montagem ARM.



Sites Web recomendados

Gavin's Guide to 80x86 Assembly: uma boa e concisa visão geral sobre linguagem de montagem x86.

The Art of Assembly Language Programming: um megalivro online de 1 500 páginas sobre o assunto. Deve ser suficiente para qualquer estudante do assunto.

Principais termos, perguntas de revisão e problemas

Principais termos

Montador (<i>assembler</i>)	Rótulo	Mnemônico
Linguagem de montagem (<i>assembly</i>)	Editor de ligação	Montador de um passo
Comentário	Ligação	Operando
Diretiva	Ligação dinâmica em tempo de carregamento	Realocação
Ligador dinâmico	Carregamento	Ligação dinâmica em tempo de execução
Instrução	Macro	Montador de dois passos

Perguntas de revisão

- B.1** Relacione alguns motivos por que vale a pena estudar programação na linguagem de montagem.
- B.2** O que é uma linguagem de montagem?
- B.3** Relacione algumas desvantagens da linguagem de montagem quando comparada a linguagens de alto nível.
- B.4** Relacione algumas vantagens da linguagem de montagem quando comparada a linguagens de alto nível.
- B.5** Quais são os elementos típicos de uma sentença da linguagem de montagem?
- B.6** Relacione e defina brevemente quatro tipos diferentes de sentenças da linguagem de montagem.
- B.7** Qual é a diferença entre um montador de um passo e um montador de dois passos?

Problemas

- B.1** Guerra de Núcleo é um jogo de programação introduzido para o público no começo dos anos 1980 (DEWDNEY, 1984ⁱ), que foi popular por um período de mais ou menos 15 anos. O Guerra de Núcleo possui quatro componentes principais: uma matriz de memória de 8 000 endereços, uma

linguagem de montagem simplificada chamada Redcode, um programa executável chamado MARS (um acrônimo para *Memory Array Redcode Simulator* – Simulador de Matriz de Memória Redcode) e um conjunto de programas de batalha que competem. Dois programas de batalha são inseridos na matriz de memória em posições escolhidas aleatoriamente; nenhum programa sabe onde está o outro. O MARS executa os programas em uma versão simples de tempo compartilhado. Dois programas se alternam: uma única instrução do primeiro programa é executada, depois uma única instrução do segundo e assim por diante. O que um programa de batalha faz durante os seus ciclos de execução depende completamente do programador. O objetivo é destruir o outro programa arruinando suas instruções. Neste problema e em vários seguintes, usamos uma linguagem ainda mais simples, chamada de CodeBlue, para explorar alguns conceitos do Guerra de Núcleos.

O CodeBlue contém apenas cinco sentenças de linguagem de montagem e usa três modos de endereçamento (Tabela B.4). Para a última posição da memória, o endereço relativo +1 se refere à primeira posição da memória. Por exemplo, `ADD #4, 6` adiciona 4 para o conteúdo da posição relativa 6 e armazena o resultado na posição 6; `JUMP @5` transfere a execução para o endereço de memória contido na posição cinco slots depois da instrução `JUMP` atual.

a. O programa `Imp` é uma única instrução `COPY 0, 1`. O que ele faz?

b. O programa `Dwarf` é a seguinte sequência de instruções:

```
ADD #4, 3
COPY 2, @2
JUMP -2
DATA 0
```

O que ele faz?

c. Reescreva `Dwarf` usando símbolos para que ele se pareça mais com um programa típico da linguagem de montagem.

B.2 O que acontece se fizermos `Imp` e `Dwarf` brigarem?

B.3 Escreva um programa de “bombardeamento” em CodeBlue que zera toda a memória (com a possível exceção das posições do programa).

B.4 Como o programa a seguir se sairia contra `Imp`?

```
Loop    COPY #0, -1
        JUMP -1
```

Dica: lembre que a execução de instruções alterna entre dois programas adversários.

B.5 a. Qual é o valor da flag `C` de estado depois da seguinte sequência:

```
mov al, 3
add al, 4
```

Tabela B.4 Linguagem de montagem CodeBlue

(a) Conjunto de instruções

Formato	Significado
DATA <value>	<value> definido na posição atual
COPY A, B	Copia origem A para destino B
ADD A, B	Adiciona A para B, colocando resultado em B
JUMP A	Transfere execução para A
JUMPZ A, B	Se B = 0, transfere para A

(b) Modos de endereçamento

Modo	Formato	Significado
Literal	# seguido de valor	Este é um modo imediato, o valor do operando está na instrução.
Relativo	Valor	O valor representa um deslocamento da posição atual, a qual contém o operando.
Indireto	@ seguido de valor	O valor representa um deslocamento da posição atual; a posição no deslocamento contém o endereço relativo da posição que contém o operando.

b. Qual é o valor da flag C de estado depois da seguinte sequência:

```
mov al, 3
sub al, 4
```

B.6 Considere a seguinte instrução NASM:

```
cmp vleft, vright
```

Para inteiros com sinal, existem três flags de estado que são relevantes. Se $vleft = vright$, então ZF é definida para 1. Se $vleft > vright$, ZF é definida para 0 e SF = OF. Se $vleft < vright$, ZF é definida para 0 e SF $\neq$ OF. Por que SF = OF se $vleft > vright$?

B.7 Considere o seguinte fragmento de código NASM:

```
mov al, 0
cmp al, al
je next
```

Escreva um programa equivalente de uma única instrução.

B.8 Considere o seguinte programa em C:

```
/*um simples programa C para encontrar média de 3 inteiros*/
main ()
{ int avg;
  int i1 = 20;
  int i2 = 13;
  int i3 = 82;
  avg = (i1 + i2 + i3)/3;
}
```

Escreva uma versão NASM deste programa.

B.9 Considere o seguinte fragmento de código em C:

```
if (EAX == 0) EBX = 1;
else EBX = 2;
```

Escreva um fragmento de código NASM equivalente.

B.10 As diretivas de inicialização de dados podem ser usadas para inicializar várias posições. Por exemplo,

```
db 0x55, 0x56, 0x57
```

reserva três bytes e inicializa seus valores.

O NASM suporta o símbolo especial \$ para permitir que cálculos envolvam a posição de montagem corrente, ou seja, \$ avalia até a posição de montagem no início da linha que contém a expressão. Tendo em mente os dois fatos anteriores, considere a seguinte sequência de diretivas:

```
message db 'hello, world'
msglen equ $-message
```

Qual é o valor atribuído ao símbolo msglen?

B.11 Suponha três variáveis simbólicas V1, V2 e V3 que contêm valores inteiros. Escreva um fragmento de código NASM que move o menor valor inteiro para ax. Use apenas as instruções mov, cmp e jbe.

B.12 Descreva o efeito desta instrução: `cmp eax, 1`

Suponha que uma instrução anterior tenha atualizado o conteúdo de eax.

B.13 A instrução xchg pode ser usada para trocar os conteúdos de dois registradores. Suponha que o conjunto de instruções x86 não suporte essa instrução.

- Implemente `xchg ax, bx` usando apenas instruções push e pop.
- Implemente `xchg ax, bx` usando apenas instrução xor (não envolva outros registradores).

B.14 No seguinte programa, suponha que a, b, x, y sejam símbolos para posições da memória principal. O que o programa faz? Você pode responder à pergunta escrevendo a lógica equivalente em C.

```

mov    eax, a
mov    ebx, b
xor    eax, x

        xor    ebx, y
        or     eax, ebx
        jnz    L2
L1:
        jmp    L3
L2:
        ; sequência de instruções...
L3:
        ; outra sequência de instruções...

```

B.15 A Seção B.1 inclui um programa C que calcula o maior divisor comum de dois inteiros.

- Descreva o algoritmo em palavras e mostre como o programa implementa a abordagem do algoritmo de Euclides para calcular o maior divisor comum.
- Adicione comentários para o programa em linguagem de máquina da Figura B.3a para deixar claro que ele implementa a mesma lógica que o programa C.
- Repita a parte (b) para o programa da Figura B.3b.

B.16 a. Um montador de dois passos pode lidar com símbolos futuros e, por isso, uma instrução pode usar um símbolo futuro como um operando. Isso nem sempre é verdadeiro para diretivas. A diretiva EQU, por exemplo, não pode usar um símbolo futuro. A diretiva 'A EQU B+1' é fácil de executar se B for definido previamente, porém é impossível se B for um símbolo futuro. Qual é razão disso?

- Sugira uma maneira para o montador eliminar esta limitação de tal forma que qualquer linha de código fonte possa usar símbolos futuros.

B.17 Considere uma diretiva com símbolo MAX que possui a seguinte forma:

símbolo MAX lista de expressões

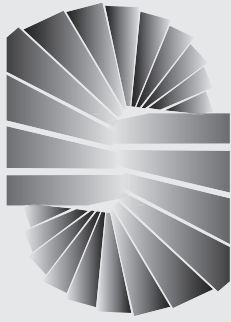
O rótulo é obrigatório e possui o valor da maior expressão no campo de operando. Exemplo:

MSGLEN MAX A, B, C ; onde A, B, C são símbolos definidos

Como MAX é executado pelo montador e em que passo?

Referências

- FOG, A. *Optimizing subroutines in assembly language: an optimization guide for x86 platforms*. Copenhagen University College of Engineering, 2008. Disponível em: <<http://www.agner.org/optimize/>>.
- CARTER, P. *PC Assembly Language*. 23 jul. 2006. Disponível no site web deste livro.
- SALOMON, D. *Assemblers and loaders*. Ellis Horwood Ltd, 1993. Disponível no site Web deste livro.
- BECK, L. *System software*. Reading, MA: Addison-Wesley, 1997.
- CLARKE, D. e MERUSI, D. *System software programming: the way things work*. Upper Saddle River, NJ: Prentice Hall, 1998.
- LEVINE, J. *Linkers and Loaders*. San Francisco: Morgan Kaufmann, 2000.
- BARTLETT, J. *Programming from the Ground Up*. 2003. Disponível no site Web deste livro.
- KNAGGS, P. e WELSH, S. *ARM: Assembly Language Programming*. Bournemouth University, School of Design, Engineering, and Computing, ago. 2004. Disponível em: <www.freetchbooks.com/arm-assembly-language-programming-t729.html>.
- DEWDNEY, A. "In the game called core war hostile programs engage in a battle of bits." *Scientific American*, mai. 1984.



Glossário

acesso direto A capacidade de obter dados de um dispositivo de armazenamento ou de entrar com dados para um dispositivo de armazenamento em uma sequência independente de sua posição relativa, por meio de endereços que indicam o local físico dos dados.

acesso direto à memória (DMA, do inglês *direct memory access*) Uma forma de E/S em que um módulo especial, chamado *módulo de DMA* (ou *controlador DMA*), controla a transferência de dados entre a memória principal e um módulo de E/S. A CPU envia uma solicitação para a transferência de um bloco de dados ao módulo de DMA e é interrompida somente depois que o bloco inteiro tiver sido transferido.

acumulador O nome de um registrador da CPU, especificado em um formato de instrução de único endereço. O acumulador, ou AC, é implicitamente um dos dois operandos da instrução.

arbitração de barramento O processo de determinar qual *bus master* concorrente terá permissão para acessar o barramento.

memória de controle Parte da memória que contém microcódigos.

memória virtual O espaço de armazenamento que pode ser considerado como armazenamento principal endereçável pelo usuário de um sistema de computação em que os endereços virtuais são mapeados para endereços reais. O tamanho da memória virtual é limitado pelo esquema de endereçamento do sistema de computação e pela quantidade de armazenamento auxiliar disponível, e não pelo número real de locais na memória principal.

arquitetura de computador Os recursos de um sistema visíveis a um programador ou, em outras palavras, os recursos que possuem um impacto direto sobre a execução lógica de um programa. Alguns exemplos de recursos da arquitetura de um computador incluem o conjunto de instruções, o número de bits (tamanho das palavras) utilizados para representar diversos tipos de dados (por exemplo, números, caracteres), mecanismos de E/S e técnicas para endereçar a memória.

array lógico programável (PLA, do inglês *programmable logic array*) Um conjunto de portas lógicas cujas interconexões podem ser programadas para realizar uma função lógica específica.

array redundante de discos independentes (RAID, do inglês *redundant array of independent disks*) Um conjunto de discos em que parte da capacidade física de armazenamento é usada para armazenar informações redundantes sobre os dados de usuário, armazenados no restante da capacidade de armazenamento. A informação redundante permite a regeneração de dados do usuário caso um dos discos membros do conjunto ou o caminho de acesso falhe.

ASCII Do inglês *American Standard Code for Information Interchange*. O ASCII é um código de 7 bits usado para representar caracteres numéricos, alfabéticos e especiais, imprimíveis. Ele também inclui códigos para *caracteres de controle*, que não são impressos ou exibidos, mas especificam alguma função de controle.

autoindexação Uma forma de endereçamento indexado em que o registrador de índice é incrementado ou decrementado automaticamente, a cada referência de memória.

barramento Um caminho de comunicações compartilhado consistindo em uma única linha ou uma coleção de linhas. Em alguns sistemas de computação, CPU, memória e componentes de E/S são conectados por um barramento comum. Como as linhas são compartilhadas por todos os componentes, somente um componente de cada vez pode transmitir com sucesso.

barramento de controle A parte de um barramento de um sistema de computação usada para a transferência de sinais de controle.

barramento de dados A parte de um barramento de um sistema de computação usada para transferência de dados.

barramento de endereço A parte de um barramento de um sistema de computação usada para transferência de um endereço. Normalmente, o endereço identifica um local da memória principal ou um dispositivo de E/S.

barramento do sistema Um barramento utilizado para interconectar os principais componentes do computador (CPU, memória, E/S).

base No sistema de numeração normalmente usado em artigos científicos, o número que é elevado à potência indicada pelo expoente e depois multiplicado pela mantissa, para determinar o número real representado (por exemplo, o número 10 na expressão $2,7 \times 10^2 = 270$).

bit Dígito binário. No sistema de numeração binário puro, um dos dígitos 0 e 1.

bit de paridade Um dígito binário anexado a um grupo de dígitos binários para fazer com que a soma de todos os dígitos binários sempre seja ímpar (paridade ímpar) ou par (paridade par).

bloco de controle de processo A manifestação de um processo em um sistema operacional. Ele é uma estrutura de dados contendo informações sobre as características e o estado do processo.

buffer Armazenamento usado para compensar a diferença na taxa de fluxo de dados, ou no tempo de ocorrência de eventos, ao transferir dados de um dispositivo para outro.

bus master Um dispositivo conectado a um barramento capaz de iniciar e controlar a comunicação no barramento.

byte Uma sequência de oito bits. Também chamado de *octeto*.

cache Uma memória rápida relativamente pequena, interposta entre uma memória maior e mais lenta, e a lógica que acessa a memória maior. A cache mantém dados acessados recentemente, e é projetada para agilizar o acesso subsequente aos mesmos dados, ou a dados com localizações adjacentes.

canal de E/S Um módulo de E/S relativamente complexo, que libera a CPU dos detalhes das operações de E/S. Um canal de E/S executa uma sequência de comandos de E/S a partir da memória principal sem a necessidade de envolvimento da CPU.

canal multiplexador Um canal projetado para operar com uma série de dispositivos de E/S simultaneamente. Diversos dispositivos de E/S podem transferir registros ao mesmo tempo, intercalando itens de dados. Ver também *canal multiplexador de byte*, *canal multiplexador de bloco*.

canal multiplexador de bloco Um canal multiplexador que intercala blocos de dados. Ver também *canal multiplexador de byte*. Compare com *canal seletor*.

canal multiplexador de byte Um canal multiplexador que intercala bytes de dados. Ver também *canal de multiplexador de bloco*. Compare com *canal seletor*.

canal seletor Um canal de E/S projetado para operar com apenas um dispositivo de E/S de cada vez. Quando o dispositivo de E/S é selecionado, um registro completo é transferido, com um byte de cada vez. Compare com *canal multiplexador de bloco*, *canal multiplexador*.

CD-ROM Do inglês *Compact Disk Read-Only Memory*. Um disco não apagável usado para armazenar dados do computador. O sistema padrão utiliza discos de 12 cm e pode manter mais de 550 MB.

ciclo de busca A parte do ciclo de instrução durante a qual a CPU busca da memória a instrução a ser executada.

ciclo de execução A parte do ciclo de instrução durante a qual a CPU realiza a operação especificada pelo *opcode* da instrução.

ciclo de instrução O processamento realizado por uma CPU para executar uma única instrução.

ciclo de interrupção A parte do ciclo da instrução durante a qual a CPU verifica as interrupções. Se uma interrupção habilitada estiver pendente, a CPU salva o estado atual do programa e retoma o processamento em uma rotina de tratamento de interrupção.

ciclo indireto A parte do ciclo de instrução durante a qual a CPU realiza um acesso à memória para converter um endereço indireto em um endereço direto.

circuito combinacional Um circuito lógico digital cujos valores de saída, em qualquer instante, dependem apenas dos valores de entrada nesse instante. Um circuito combinacional é um caso especial de um circuito sequencial, que não tem uma capacidade de armazenamento. Sinônimo de *circuito combinatório*.

circuito integrado (CI) Um pequeno pedaço de material sólido, como o silício, no qual é gravado ou impresso um conjunto de componentes eletrônicos e suas interconexões.

circuito sequencial Um circuito lógico digital cuja saída depende da entrada atual e do estado do circuito. Os circuitos sequenciais, portanto, possuem o atributo de memória.

cluster Um grupo de computadores interconectados, completos, trabalhando juntos como um recurso de computação unificado, que pode criar a ilusão de ser uma única máquina. O termo *computador completo* significa um sistema que pode funcionar por conta própria, separado do *cluster*.

código de condição Um código que reflete o resultado de uma operação anterior (por exemplo, aritmética). Uma CPU pode incluir um ou mais códigos de condição, que podem ser armazenados separadamente dentro da CPU ou como parte de um registrador de controle maior. Também conhecido como *flag*.

código de correção de erro Um código em que cada caractere ou sinal se ajusta a regras específicas da construção, de modo que a quebra dessas regras indica a presença de um erro e que alguns ou todos os erros detectados podem ser corrigidos automaticamente.

código de detecção de erro Um código em que cada caractere ou sinal se ajusta a regras específicas da construção, de modo que a quebra dessas regras indica a presença de um erro.

Código de operação Código usado para representar as operações do computador. Normalmente abreviado para *opcode*.

Compact disk (CD) Um disco não apagável que armazena informações de áudio digitalizado.

componente em estado sólido Um componente cuja operação depende do controle de fenômenos elétricos ou magnéticos nos sólidos (por exemplo, transistor, diodo a cristal, núcleo de ferrite).

computação de alto desempenho (HPC, do inglês *high-performance computing*) Uma área de pesquisa que lida com supercomputadores e o software que é executado neles. A ênfase é nas aplicações científicas, que podem envolver um uso pesado de cálculos de vetores e matrizes e algoritmos paralelos.

comunicação de dados Transferência de dados entre dispositivos. O termo geralmente exclui a E/S.

conjunto de instruções do computador Um conjunto completo dos operadores das instruções de um computador, junto com uma descrição dos tipos de significados que podem ser atribuídos aos seus operandos. Sinônimo de *conjunto de instruções de máquina*.

contador de programa (PC, do inglês *program counter*) Registrador de endereço da instrução.

controlador de E/S Um módulo de E/S relativamente simples, que requer o controle da CPU ou de um canal de E/S. Sinônimo de *controlador de dispositivo*.

CPU microprogramada Uma CPU cuja unidade de controle é implementada usando microprogramação.

daisy chain Um método de interconexão de dispositivo para determinar a prioridade de interrupção conectando as fontes, que geram a interrupção, em série.

decodificador Um dispositivo que tem um número de linhas de entrada das quais qualquer número de linhas pode transportar sinais e um número de linhas de saída das quais não mais do que uma pode transportar um sinal, havendo uma correspondência um-para-um entre as saídas e as combinações de sinais de entrada.

envio de instrução O processo de iniciar a execução da instrução nas unidades funcionais do processador. Isso ocorre quando uma instrução se movimenta do estágio de decodificação do pipeline para o primeiro estágio de execução do pipeline.

disco magnético Um prato circular com uma camada de superfície magnetizável, em um ou em ambos os lados, nos quais os dados podem ser armazenados.

disco óptico apagável Um disco que usa tecnologia óptica, mas que pode ser facilmente apagado e regravado. Estão em uso discos de 3,25 polegadas e de 5,25 polegadas. Uma capacidade típica é de 650 MB.

disquete Um disco magnético flexível envolvido em um recipiente protetor. Sinônimo de *disco flexível*.

E/S controlada por interrupção Uma forma de E/S. A CPU emite um comando de E/S, continua a executar instruções subsequentes e é interrompida pelo módulo de E/S quando seu trabalho estiver completo.

E/S isolada Um método de endereçar módulos de E/S e dispositivos externos. O espaço de endereços de E/S é tratado separadamente do espaço de endereços da memória principal. Instruções de máquina específicas de E/S precisam ser usadas. Compare com *E/S mapeada na memória*.

E/S mapeada na memória Um método de endereçamento de módulos de E/S e dispositivos externos. Um único espaço de endereços é usado para a memória principal e para os endereços de E/S, e as mesmas instruções de máquina são usadas para leitura/escrita da memória e de E/S.

E/S programada Uma forma de E/S em que a CPU emite um comando de E/S a um módulo de E/S e necessita esperar o término da operação, antes de prosseguir seu processamento.

emulação A imitação de todo ou parte de um sistema por outro, principalmente pelo hardware, de modo que o sistema que está imitando aceita os mesmos dados, executa os mesmos programas e alcança os mesmos resultados do sistema imitado.

endereço absoluto Um endereço em uma linguagem de computação que identifica um local de armazenamento ou um dispositivo, sem o uso de qualquer referência intermediária.

endereço base Um valor numérico que é usado como uma referência no cálculo dos endereços, na execução de um programa de computador.

endereço direto Um endereço que designa o local de armazenamento de um item de dados a ser tratado como operando. Sinônimo de *endereço de nível um*.

endereço imediato Conteúdo de um campo de endereço que contém o valor do operando e não o endereço do operando. Sinônimo de *endereço de nível zero*.

endereço indexado Um endereço que é modificado pelo conteúdo de um registrador de índice antes ou durante a execução de uma instrução do computador.

endereço indireto Um endereço de um local de armazenamento que contém outro endereço.

entrada/saída (E/S) Pertencente a entrada, saída ou ambos. Refere-se à movimentação de dados entre um computador e um periférico ligado diretamente.

equipamento periférico Em um sistema de computação, com relação a uma unidade de processamento em particular, qualquer equipamento que oferece comunicação externa para a unidade de processamento. Sinônimo de *dispositivo periférico*.

escalar Uma quantidade caracterizada por um único valor.

espaço de endereçamento O intervalo de endereços (memória ou E/S) que pode ser referenciado.

execução especulativa A execução das instruções ao longo de um caminho de um desvio. Se mais tarde esse desvio não for tomado, então os resultados da execução especulativa são descartados.

execução prevista Um mecanismo que admite a execução condicional de instruções individuais. Isso possibilita a execução especulativa de desvios de uma instrução de desvio, e manutenção dos resultados do último desvio que foi tomado.

falta de página Ocorre quando uma página contendo uma palavra referenciada não está na memória principal. Isso causa uma interrupção e exige que o sistema operacional traga a página necessária.

firmware Microcódigo armazenado em uma memória somente de leitura.

fita magnética Uma fita com uma camada de superfície magnetizável, onde os dados podem ser armazenados por gravação magnética.

flip-flop Um circuito ou dispositivo contendo elementos ativos, capazes de assumir um ou dois estados estáveis em um determinado momento. Sinônimo de *circuito biestável*, *toggle*.

formato de instrução O layout de uma instrução de computador como uma sequência de bits. O formato divide a instrução em campos, correspondentes aos elementos constituintes da instrução (por exemplo, *opcode*, operandos).

frame de página Uma área do armazenamento principal usada para manter uma página.

G Prefixo significando 2^{30} .

indexação Uma técnica de modificação de endereço por meio de registradores de índice.

instrução de computador Uma instrução que pode ser reconhecida pela unidade de processamento do computador para o qual ela foi projetada. Sinônimo de *instrução de máquina*.

interrupção Uma suspensão de um processo, como a execução de um programa do computador, causada por um evento externo a esse processo, e realizada de modo que o processo possa ser retomado.

interrupção desabilitada Uma condição, normalmente criada pela CPU, durante a qual esta ignora os sinais de solicitação de interrupção de uma classe especificada.

interrupção habilitada Uma condição, normalmente criada pela CPU, durante a qual esta responde a sinais de solicitação de interrupção de uma classe especificada.

K Prefixo significando $2^{10} = 1.024$. Assim, 2Kb = 2.048 bits.

linguagem de montagem (linguagem assembly) Uma linguagem orientada ao computador cujas instruções normalmente têm correspondência um-para-um com as instruções do computador, e que pode oferecer facilidades como o uso de macroinstruções. Sinônimo de *linguagem dependente do computador*.

linguagem de microprogramação Um conjunto de instruções usado para especificar microprogramas.

linha de cache Um bloco de dados que constitui a unidade de transferência entre cache e memória. A ela está associada uma *tag*, que a identifica na memória cache.

localidade de referência A tendência de um processador de acessar o mesmo conjunto de locais de memória repetidamente por um curto período.

M Prefixo significando $2^{20} = 1.048.576$. Assim, 2 Mb = 2.097.152 bits.

mainframe Um termo originalmente referenciando o gabinete que contém a unidade central de processamento ou “*main frame*” de uma grande máquina de processamento em lotes (*batch*). Após o surgimento do projeto de minicomputadores, no início da década de 1970, as máquinas tradicionais maiores foram descritas como computadores mainframe, ou mainframes. As características típicas de um mainframe são que ele suporta um banco de dados grande, possui um hardware de E/S elaborado e é usado em uma instalação de processamento de dados central.

memória associativa Uma memória cujos locais de armazenamento são identificados por seu conteúdo, ou por uma parte de seu conteúdo, em vez de suas posições.

memória cache É um buffer especial para armazenamento, menor e mais rápido que o armazenamento principal, usado para manter uma cópia de instruções e dados, obtidos do armazenamento principal, que provavelmente serão utilizados em seguida, pelo processador.

memória de acesso aleatório (RAM, do inglês *random-access memory*) Memória em que cada local endereçável tem um mecanismo de endereçamento. O tempo para acessar um determinado local independe da sequência de acesso anterior.

memória não volátil Memória cujo conteúdo é estável e não exige uma fonte de alimentação constante.

memória principal Armazenamento endereçável pelo programa, do qual as instruções e outros dados podem ser carregados diretamente em registradores para subsequente execução ou processamento.

memória secundária Memória localizada fora do próprio sistema de computação, ou seja, ela não pode ser processada diretamente pelo processador. Ela primeiro deve ser copiada para a memória principal. Alguns exemplos incluem discos e fitas.

memória somente de leitura (ROM, do inglês *read-only memory*) Memória semicondutora cujo conteúdo não pode ser alterado, exceto destruindo a unidade de armazenamento. Memória não apagável.

memória somente de leitura programável (PROM, do inglês *programmable read-only memory*) Memória semicondutora cujo conteúdo pode ser definido apenas uma vez. O processo de escrita é realizado eletricamente e pode ser realizado pelo usuário em um momento posterior à fabricação original do chip.

memória volátil Uma memória em que uma fonte de alimentação elétrica constante é necessária para manter o conteúdo da memória. Se a energia for desligada, a informação armazenada é perdida.

microcomputador Um sistema de computador cuja unidade de processamento é um microprocessador. Um microcomputador básico inclui um microprocessador, armazenamento e uma facilidade para entrada/saída, que podem ou não estar em um único chip.

microinstrução Uma instrução que controla o fluxo e a sequência de dados em um processador em um nível mais fundamental do que as instruções de máquina. Instruções de máquina individuais e talvez outras funções podem ser implementadas por microprogramas.

micro-operação Uma operação elementar da CPU, realizada durante um pulso de clock.

microprocessador Um processador cujos elementos foram minimizados para um ou alguns circuitos integrados.

microprograma Uma sequência de microinstruções que estão em um armazenamento especial, onde podem ser acessadas dinamicamente para realizar diversas funções.

módulo de E/S Um dos principais tipos de componente de um computador. Ele é responsável pelo controle de um ou mais dispositivos externos (periféricos) e pela troca de dados entre esses dispositivos e a memória principal e/ou registradores da CPU.

multiplexador Um circuito combinatório que conecta múltiplas entradas a uma única saída. A qualquer momento, apenas uma das entradas é selecionada para ser passada para a saída.

multiprocessador Um computador que tem dois ou mais processadores, que possuem acesso comum a um armazenamento principal.

multiprocessador de acesso não uniforme à memória (NUMA, do inglês *nonuniform memory access*) Um multiprocessador de memória compartilhada, em que o tempo de acesso para determinado processador a uma palavra na memória varia com o local desta palavra na memória.

multiprocessamento simétrico (SMP, do inglês *symmetric multiprocessing*) Uma forma de multiprocessamento que permite que o sistema operacional seja executado em qualquer processador disponível ou em diversos processadores disponíveis, simultaneamente.

multiprogramação Um modo de operação que permite a execução intervalada de dois ou mais programas de computador por um único processador.

multitarefa Um modo de operação que oferece desempenho concorrente ou execução intervalada de duas ou mais tarefas do computador. O mesmo que multiprogramação, usando uma terminologia diferente.

núcleo (kernel) A parte de um sistema operacional que contém suas funções básicas e usadas com mais frequência. Normalmente o núcleo permanece residente na memória principal.

opcode Forma abreviada para *operation code*. Ver *código operacional*.

operador binário Um operador que representa uma operação sobre dois e somente dois operandos.

operador unário Um operador que representa uma operação sobre um e somente um operando.

operando Uma entidade sobre a qual uma operação é realizada.

organização do computador Refere-se às unidades operacionais e suas interconexões, que realizam as especificações da arquitetura do computador. Os recursos da organização do computador incluem os detalhes do hardware transparentes ao programador, tais como sinais de controle, interfaces entre o computador e periféricos e a tecnologia de memória utilizada.

ortogonalidade Um princípio pelo qual duas variáveis ou dimensões são independentes umas das outras. No contexto de um conjunto de instruções, o termo geralmente é usado para indicar que outros elementos de uma instrução (modo de endereçamento, número de operandos, tamanho do operando) são independentes do (não determinados pelo) *opcode*.

pack de discos Uma montagem de discos magnéticos que podem ser removidos como um todo de uma unidade de disco, junto com um recipiente do qual a montagem deverá ser separada quando estiver operando.

página Em um sistema de armazenamento virtual, um bloco de tamanho fixo que tem um endereço virtual e que é transferido como uma unidade entre o armazenamento real e o armazenamento auxiliar.

paginação por demanda A transferência de uma página do armazenamento auxiliar para o armazenamento real, quando necessário.

palavra (word) Um conjunto ordenado de bytes ou bits, que é a unidade normal em que a informação pode ser armazenada, transmitida ou operada dentro de determinado computador. Normalmente, se um processador tem um conjunto de instruções de tamanho fixo, então o tamanho da instrução é igual ao tamanho da palavra.

palavra de instrução muito longa (VLIW, do inglês *very long instruction word*) Refere-se ao uso de instruções que contêm múltiplas operações. Com efeito, múltiplas instruções estão contidas em uma única palavra. Normalmente, uma VLIW é construída pelo compilador, que coloca operações que possam ser executadas em paralelo na mesma palavra.

palavra de estado do programa (PSW, do inglês *program status word*) Uma área no armazenamento usada para indicar a ordem em que as instruções são executadas, e para manter e indicar o estado do sistema de computação. Sinônimo de *palavra de estado do processador*.

pilha Uma lista ordenada em que os itens são inseridos e excluídos da mesma extremidade da lista, conhecida como topo. Ou seja, o próximo item inserido à lista é colocado no topo, e o próximo item a ser removido da lista é o item que estava na lista por menos tempo. Esse método é caracterizado como “último a entrar, primeiro a sair” (*last-in-first-out*).

pipeline Uma organização de processador em que o processador consiste de uma série de estágios, permitindo que múltiplas instruções sejam executadas simultaneamente.

porta lógica Um circuito eletrônico que produz um sinal de saída que é uma operação booleana simples de seus sinais de entrada.

previsão de desvio (*branch prediction*) Um mecanismo utilizado pelo processador para prever o resultado de um desvio condicional do programa, antes de sua execução.

processador Em um computador, uma unidade funcional que interpreta e executa instruções. Um processador consiste em pelo menos uma unidade de controle de instrução e uma unidade aritmética.

processador de E/S Um módulo de E/S com seu próprio processador, capaz de executar suas próprias instruções, específicas, de E/S ou, em alguns casos, instruções de máquina de uso geral.

processador superescalar Um projeto ou processador que inclui pipelines de múltiplas instruções, de modo que mais de uma instrução pode ser executada no mesmo estágio de pipeline simultaneamente.

processador com superpipeline Um projeto de processador em que o pipeline de instrução consiste em muitos estágios pequenos, de modo que mais de um estágio de pipeline pode ser executado durante um ciclo de clock e, com isto, um grande número de instruções pode estar no pipeline ao mesmo tempo.

processo Um programa em execução. Um processo é controlado e escalonado pelo sistema operacional (SO).

protocolo de coerência de cache Um mecanismo para manter a validade dos dados entre múltiplas caches, de modo que cada acesso aos dados sempre adquira a versão mais recente do conteúdo de uma palavra da memória principal.

RAM dinâmica Uma memória RAM cujas células são implementadas usando capacitores. Uma RAM dinâmica perderá seus dados gradualmente, a menos que os dados sejam reescritos periodicamente (*refresh* de memória).

RAM estática Uma memória RAM cujas células são implementadas por meio de flip-flops. Uma RAM estática manterá seus dados enquanto houver energia nela; os dados não necessitam ser reescritos periodicamente (*refresh* de memória).

registrador de buffer de memória (MBR, do inglês *memory buffer register*) Um registrador que contém os dados lidos da memória ou os dados a serem escritos na memória.

registrador de endereço de instrução Um registrador de uso especial, utilizado para manter o endereço da próxima instrução a ser executada.

registrador de endereço de memória (MAR, do inglês *memory address register*) Um registrador, em uma unidade de processamento, que contém o endereço do local de memória que está sendo acessado.

registrador de índice Um registrador cujo conteúdo pode ser usado para modificar um endereço de operando durante a execução de instruções; ele também pode ser usado como um contador. Um registrador de índice pode ser usado para controlar a execução de um loop, para controlar o uso de um *array*, como uma chave, para pesquisa em uma tabela de pesquisa ou como um ponteiro.

registrador de instrução Um registrador que é usado para manter uma instrução para interpretação.

registrador de propósito geral Um registrador, normalmente endereçável explicitamente, dentro de um conjunto de registradores, que pode ser usado para diferentes finalidades como, por exemplo, um acumulador, um registrador de índice ou um manipulador especial de dados.

registradores A memória de alta velocidade, interna à CPU. Alguns registradores são visíveis ao usuário, ou seja, estão disponíveis ao programador por meio do conjunto de instruções da máquina. Outros registradores são usados apenas pela CPU, para fins de controle.

registradores de controle Registradores da CPU empregados para controlar a operação desta. A maioria deles não é visível ao usuário.

registradores visíveis ao usuário Registradores da CPU que podem ser referenciados pelo programador. O formato do conjunto de instruções permite que um ou mais registradores sejam especificados como operandos ou endereços de operandos.

representação em complemento de um Usada para representar inteiros binários. Um inteiro positivo é representado como na representação sinal-magnitude. Um inteiro negativo é representado pela reversão de cada bit, na representação de um inteiro positivo de mesma magnitude.

representação em complemento de dois Usada para representar inteiros binários. Um inteiro positivo é representado como na representação sinal-magnitude. Um número negativo é representado calculando o complemento booleano de cada bit do número positivo correspondente, somando, depois, 1 ao padrão de bits resultante, considerando o padrão de bits resultante como um inteiro sem sinal.

representação sinal-magnitude Usada para representar inteiros binários. Em uma palavra de N bits, o bit mais à esquerda é o sinal (0 = positivo, 1 = negativo) e os $N - 1$ bits restantes compreendem a magnitude do número.

salto condicional Um salto que ocorre somente quando a instrução que o especifica é executada e as condições especificadas são satisfeitas. Compare com *salto incondicional*.

salto incondicional Um salto que ocorre sempre que a instrução que o especificou é executada.

semicondutor Uma substância sólida cristalina, como o silício ou o germânio, cuja condutividade elétrica é intermediária entre os isolantes e os bons condutores. Usado para fabricar transistores e componentes em estado sólido.

sistema de representação de ponto fixo Na representação em ponto fixo, sistema em que a posição da vírgula da raiz (binário ou decimal, ou octal, ou hexadecimal) é fixa para um fator de escala (expoente) implícito (não representado) e constante.

sistema de representação de ponto flutuante Um sistema de numeração em que um número real é representado por um par de numerais distintos: o primeiro é o produto da parte de ponto fixo; o segundo é um valor obtido elevando-se a base de ponto flutuante, implícita, a uma potência indicada pelo expoente da representação de ponto flutuante.

sistema operacional (SO) Software que controla a execução dos programas e que oferece serviços como alocação de recursos, escalonamento, controle de entrada/saída e gerenciamento de dados.

stripping de disco Um tipo de mapeamento de um conjunto de disco em que blocos de dados logicamente contíguos, ou *strips*, são mapeados em um padrão *round-robin* a membros consecutivos do array. Um conjunto de *strips* logicamente consecutivos que mapeia exatamente um *strip* a cada membro do conjunto é conhecido como um *stripe*.

tabela verdade Uma tabela que descreve uma função lógica listando todas as combinações possíveis de valores de entrada e indicando, para cada combinação, o valor de saída.

tempo de ciclo da memória O inverso da taxa em que a memória pode ser acessada. Esse é o tempo mínimo entre a resposta a uma solicitação de acesso (leitura ou escrita) e a resposta à próxima solicitação de acesso.

tempo de ciclo de processador O tempo exigido para a menor micro-operação bem definida da CPU. Ela é a unidade de tempo básica para medir todas as ações da CPU. Sinônimo de *tempo de ciclo de máquina*.

temporização assíncrona Uma técnica em que a ocorrência de um evento em um barramento acompanha e depende da ocorrência de um evento anterior.

temporização síncrona Uma técnica em que a ocorrência de eventos em um barramento é determinada por um clock. Um clock define *slots* de tempo de mesma largura, e os eventos começam apenas no início de um *slot* de tempo.

unidade central de processamento (UCP ou CPU, do inglês *central processing unit*) A parte de um computador que busca e executa instruções. Ela consiste em uma unidade lógica e aritmética (ALU), uma unidade de controle e registradores. Normalmente chamada apenas de *processador*.

unidade de controle A parte da CPU que controla suas operações, incluindo as operações da ALU, a movimentação de dados dentro da CPU e a troca de dados e de sinais de controle por meio de interfaces externas (por exemplo, o barramento do sistema).

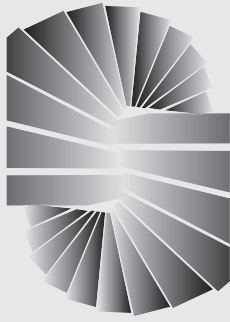
unidade lógica e aritmética (ULA ou ALU, do inglês *arithmetic logic unit*) Uma parte de um computador que realiza operações aritméticas e lógicas.

uniprocessamento Execução sequencial de instruções por uma unidade de processamento, ou uso independente de uma unidade de processamento, em um sistema de multiprocessamento.

variável global Uma variável definida em uma parte de um programa de computador e usada em pelo menos uma outra parte desse mesmo programa.

variável local Uma variável que é definida e usada apenas em uma parte especificada de um programa de computador.

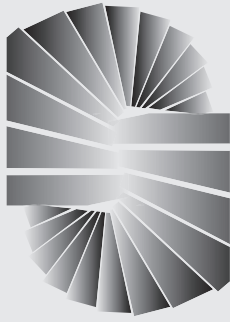
vetor (vector ou *array*) Uma quantidade normalmente caracterizada por um conjunto ordenado de escalares.



Referências

- ANDERSON, D.; SPARACIO, E. e TOMASULO, F. "The IBM System/360 Model 91: machine philosophy and instruction handling". *IBM journal of research and development*, jan. 1967.
- BELL, C.; CADY, R.; MCFARLAND, H.; DELAGI, B.; O'LOUGHLIN, J. e NONAN, R. "A new architecture for minicomputers — the DEC PDP-11". *Proceedings, spring joint computer conference*, 1970.
- BHARANDWAJ, J. et al. "The Intel IA-64 compiler code generator". *IEEE Micro*, set./out. 2000.
- BROWN, S. e ROSE, S. "Architecture of FPGAs and CPLDs: a tutorial". *IEEE design and test of computers*, vol. 13, n. 2, 1996.
- CHASIN, A. "Predication, speculation, and modern CPUs". *Dr. Dobbs's journal*, mai. 2000.
- COLWELLI, R.; HITCHCOCK, C.; JENSEN, E. e SPRUNT, H. "More controversy about 'computers, complexity, and controversy'". *Computer*, dez. 1985.
- DULONG, C. "The IA-64 architecture at work". *Computer*, jul. 1998.
- ECKERT, R. "Communication between computers and peripheral devices — an analogy". *ACM SIGCSE Bulletin*, set. 1990.
- EVANS, J. e TRIMPER, G. *Itanium architecture for programmers*. Upper Saddle River, NJ: Prentice Hall, 2003.
- FURHT, B. e MILUTINOVIC, V. "A survey of microprocessor architectures for memory management". *Computer*, mar. 1987.
- FUTRAL, W. *InfiniBand architecture: development and deployment*. Hillsboro, OR: Intel Press, 2001.
- GOLDBERG, D. "What every computer scientist should know about floating-point arithmetic". *ACM Computing surveys*, mar. 1991.
- GREGG, J. *Ones and zeros: understanding boolean algebra, digital circuits, and the logic of sets*. Nova York: Wiley, 1998.
- HALFHILL, T. "Beyond Pentium II". *Byte*, dez. 1997.
- HAEUSSER, B. et al. *IBM system storage tape library guide for open systems*. IBM Redbook SG24-5946-05, out. 2007. Disponível em <ibm.com/redbooks>.
- HENNING, J. "SPEC CPU2006 benchmark descriptions". *Computer architecture news*, set. 2006.
- HINTON, G. et al. "The microarchitecture of the Pentium 4 processor". *Intel technology journal*, Q1 2001. Disponível em <<http://developer.intel.com/technology/itj/>>.
- HUCK, J. et al. "Introducing the IA-64 architecture". *IEEE Micro*, set./out. 2000.
- HWU, W. "Introduction to predicated execution". *Computer*, jan. 1998.
- HWU, W.; AUGUST, D. e SIAS, J. "Program decision logic optimization using predication and control speculation". *Proceedings of the IEEE*, nov. 2001.
- INTEL CORP. *Intel IA-64 Architecture Software Developer's Manual (4 volumes)*. Documentos 245317 a 245320. Aurora, CO, 2000.
- INTEL CORP. *Itanium processor microarchitecture reference for software optimization*. Aurora, CO, Documento 245473, ago. 2000.

- JARP, S. "Optimizing IA-64 Performance". *Dr. Dobbs's journal*, jul. 2001.
- JOUPPI, N. "The nonuniform distribution of instruction-level and machine parallelism and its effect on performance". *IEEE transactions on computers*, dez. 1989.
- KAGAN, M. "InfiniBand: thinking outside the box design". *Communications system design*, set. 2001. Disponível em www.csdmag.com.
- KATHAIL, B.; SCHLANSKER, M. e RAU, B. "Compiling for EPIC architectures". *Proceedings of the IEEE*, nov. 2001.
- LEONG, P. "Recent trends in FPGA architectures and applications". *Proceedings, 4th IEEE international symposium on electronic design, test, and applications*, 2008.
- MAHLKE, S. et al. "A comparison of full and partial predicated execution support for ILP processors". *Proceedings, 22nd international symposium on computer architecture*, jun. 1995.
- MAHLKE, S. et al. "Characterizing the impact of predicated execution on branch prediction". *Proceedings, 27th international symposium on micro architecture*, dez. 1994.
- MAZIDI, M. e MAZIDI, J. *The 80x86 IBM PC and compatible computers: assembly language, design and interfacing*. Upper Saddle River, NJ: Prentice Hall, 2003.
- McNAIRY, C. e SOLTIS, D. "Itanium 2 processor microarchitecture". *IEEE Micro*, mar.-abr. 2003.
- NAFFZIGER, S. et al. "The implementation of the itanium 2 microprocessor". *IEEE journal of solid-state circuits*, nov. 2002.
- PATTERSON, D. e HENNESSY, J. "Response to 'computers, complexity, and controversy'". *Computer*, nov. 1985.
- SCHLANSKER, M. e RAU, B. *EPIC: an architecture for instruction-level parallel processors*. HPL Technical Report HPL-1999-111, Hewlett-Packard Laboratories (www.hpl.hp.com), fev. 2000.
- SCHLANSKER, M. e RAU, B. "EPIC: explicitly parallel instruction computing". *Computer*, fev. 2000.
- SEGARS, S.; CLARKE, K. e GOUDGE, L. "Embedded control problems, thumb, and the ARM7TDMI". *IEEE Micro*, out. 1995.
- SHANNON, C. "Symbolic analysis of relay and switching circuits". *AIEE transactions*, vol. 57, 1938.
- SHANLEY, T. *InfinBand network architecture*. Reading, MA: Addison-Wesley, 2003.
- SHARANGPANI, H. e ARONA, K. "Itanium processor microarchitecture". *IEEE Micro*, set./out. 2000.
- SODERQUIST, P. e LEESER, M. "Area and performance tradeoffs in floating-point divide and square-root implementations". *ACM computing surveys*, set. 1996.
- STALLINGS, W. *Data and computer communications, eighth edition*. Upper Saddle River, NJ: Prentice Hall, 2007.
- STENSTROM, P. "A survey of cache coherence schemes of multiprocessors". *Computer*, jun. 1990.
- STONHAM, T. *Digital logic techniques*. Londres: Chapman & Hall, 1996.
- THOMPASON, D. "IEEE 1394: changing the way we do multimedia communications". *IEEE multimedia*, abri.-jun. 2000.
- TRIEBEL, W. *Itanium architecture for software developers*. Intel Press, 2001.
- WILKES, M. "Slave memories and dynamic storage allocation". *IEEE transactions on electronic computers*, abr. 1965. Reimpresso em Hill, 2000.



Índice

A

Acerto/falha, leitura/escrita com, 526-527
Acesso aleatório, 91
Acesso direto, 91
Acesso Direto à Memória (DMA), 68, 176, 191-193, 195-196
 configurações, 193, 194
 estrutura de interconexão, 67
 função, 176, 191-193
 Intel 8237A, controlador, 193, 195-196
 registradores, 195-196
 roubo de ciclos (*cycle stealing*), 192-193
Acesso sequencial, 91
Acumulador (AC), 16, 57, 290
Adição, 256-258, 273-275
 inteiros na representação complemento de dois, 256-258
 números na representação com (ou em) ponto flutuante, 273-275
Algoritmo de Dijkstra, 323-325
Algoritmos de substituição, memória cache, 109-110
Alocação, processador Pentium 4, 445
Alocação de bits, 340-343
American Standard Code for Information Interchange (ASCII), 179
Antidependência, 437
Arbitração, 72-73, 82-83
 método de interconexão, 72-73
 PCI, 82-83
Árbitro (controlador de barramento), 72-73
Aritmética de computador. *Ver* unidade aritmética e lógica - ALU
Aritmética de saturação, 313
ARM (*Advanced RISC Machine*), 1-2, 35-38, 115-116, 238-243, 296, 313-315, 337-339, 348-349, 385-389, 446-452, 571-574
ARM11 MPCore, 571-575
 códigos de condição, 314-315
 conjuntos de instruções da CPU, 296, 313-315, 337-339, 348-349

 controle de acesso, 242-243
 Cortex-A8, processador, 446-452
 Current Program Status Registers (CPSR), 387-389
 evolução, 36-38
 formato de instrução, 348-349
 formatos para gerenciamento de memória, 240-243
 gerenciamento de memória, 238-243
 instruções de máquina, 296, 313-315
 memória cache, 115-116
 modos, 386-387
 modos de endereçamento, 337-339
 MPCore (computadores multicore), 571-575
 operações (opcode), 313-315
 organização do processador, 385-389
 organização do registrador, 387-389
 organização do sistema de memória, 238-239
 paralelismo em nível de instrução, 446-452
 parâmetros para gerenciamento de memória, 242
 processamento de interrupção, 389
 projeto de processador superescalar, 446-452
 sistemas embarcados, 35-38
 tabelas de página, 239-240
 tipos de dados, 296
 tradução de endereço de memória virtual, 239-240
 Translation Lookaside Buffer (TLB), 238-239
 Unidade de Gerenciamento de Memória (MMU), 239-240
Armazenamento de variável global, registradores, 402
Arquitetura, *InfiniBand*, 203
Arredondamento, padrões IEEE, 277-279
Atraso rotacional (latência), discos magnéticos, 156
Autoindexação, 36

B

Balanceamento de carga, clusters, 536
Banco de memória, 135
Barramento de microinstruções (MIB), 496-497, 498
Big endian, ordenação, 325-329
Bit de modificação (uso), 110
bits de guarda, 276-277
Bits de paridade, 137
Blu-ray DVD, 166, 169
Booleanas (lógicas), instruções, 289
Booth, algoritmo, 262-265
Borda de descida, 87-88
Borda de subida, 87-88
Buffer de alvo do desvio (BTB), 443-445, 448-449
Buffer de histórico global (GHB), 448
Buffer de laço, pipeline, 373
Buffering de dados, módulos de E/S, 180-181
Busca antecipada, 365, 372-373
Busca na cache de trace, processador Pentium 4, 443-445

C

Cabeças de discos magnéticos, 150, 153
Cache, 30
Cache DRAM, 145
Cache-Coherent Nonuniform Memory Access (Acesso não uniforme à memória - CC-*NUMA*), 538-539
Camada de enlace, 200-202, 205
Camada de rede, 205
Camada de transporte, 205
Camada física, 200-201, 205
Campos de endereço virtual, 234-235
Canal multiplexador, 198
Canal seletor, 196-197
CAR (*Control address register*), 487
Chamadas de procedimento, 305-307, 309, 399
 implementação de pilha, 306-307
 instruções de transferência de controle, 303-304
 Intel x86, instruções de chamada/retorno, 309-310

- linguagem de programação de alto nível (*high-level language* – HLL), 399-400
- Reduced instruction set computers* (RISC), 399-400
- Chip de multiprocessamento, 514, 531-532. *Ver também* MPCore (computadores multicore)
- IBM Power5, 532
- multithreading* e, 514, 531-532
- Pentium 4, processador, 532
- Chips, 26-27, 31-34, 132-135
 - células de memória principais, 132-135
 - Ciclo de busca, 16, 56-59, 361, 362, 463-464
 - Ciclo de execução, 17, 56-59, 361, 363, 465-466
 - desenvolvimento, 26-27
 - DRAM, 140-145
 - empacotamento, 134
 - EPROM, pacote, 134
 - instruções de computador, 17, 56-59
 - instruções do computador, 16, 56-59
 - instruções do processador, 361, 362
 - lógicos, 132-133
 - memória intercalada (banco de memória), 135-136
 - memória interna, 132-135
 - micro-operações (micro-ops), 463-466
 - instrução do processador, 361, 363
 - organização, 31-34, 134-135
 - pino *Chip Enable* (CE), 134
 - processamento do computador, 31-34
- Ciclo de instrução, 15-16, 56-66, 361-364, 466. *Ver também* acesso direto à memória (DMA)
 - ciclo de busca, 16, 56-59, 361, 363
 - ciclo de execução, 17, 56-59, 361, 363
 - ciclo de interrupção, 60-63, 361, 363
 - ciclo indireto, 362
 - código (ICC), 466
 - computador IAS, 15-17
 - diagramas de estado, 59, 64, 363
 - fluxo de dados, 362-363
 - funções do computador, 56-66
 - interrupções, 59-66
 - micro-operações, 465-466
 - módulos de E/S, 67
 - múltiplas interrupções, 63-66
 - processador, 357-361
- Ciclo de instrução indireto, 362, 363, 464
- Ciclo de interrupção, 59-66, 361, 363, 465
 - instruções de processador, 361, 363
 - instruções do computador, 60-63, 64
 - micro-operações (micro-ops), 464-466
- Ciclos de máquina, 474-475
- Cilindro, 153
- Circuitos integrados, desenvolvimento, 21-26
- CISC, 2, 396, 398, 406-410, 424-425
 - características *versus* RISC, 409-410, 424-425
 - High-level language* (linguagens de programação de alto nível - HLL) e, 398, 406-407
 - sistemas superescalares e RISC, comparação com, 396
- Clock* (barramento), ciclo, 73-74
- Clusters*, 514, 532-538
 - arquitetura computacional, 536-537
 - configurações, 534-535
 - projeto dos sistemas operacionais, 536
 - requisitos de projeto, 532-533
 - secundário passivo (*standby passive*), 534-535
 - secundários ativos, 534-535
 - servidores *blade*, 537-538
 - SMP em comparação com, 538
 - técnica de disco compartilhado, 535
 - técnica sem compartilhamento, 535
- Codificação, execução de microinstrução, 495-496
- Codificação de recurso, 495-496
- Codificação funcional, 495
- Código de Hamming, 137
- Códigos de condição, 303-304, 310, 314-315, 357-358
 - ARM, 314-315
 - instruções de desvio, 303-304
 - Instruções SIMD x86, 310
 - registradores, 357-358
- Coerência de cache, 515, 523-526, 573-575
 - acerto/falha de escrita, 526-527
 - acerto/falha de leitura, 526-527
 - consistência de cache L1-L2, 527
 - MPCore (computadores multicore), 573-575
 - problema, 523
 - processador paralelo, 514, 523-527
 - protocolo MESI (*Shared*), 525-527
 - protocolos de diretório, 524
 - protocolos de monitoração (*snoopy*), 524-525
 - soluções por hardware, 524
 - soluções por software, 523-524
- Comandos, 79-80, 182
 - E/S programada, 181-182
 - Peripheral component interconnection* (PCI), 79-80
- Compact Disk* (CD), 164-168
- Digital Versatile Disk* (DVD), comparação com, 168
 - gravável (CD-R), 166, 168
 - regravável (CD-RW), 166, 168
 - somente de leitura (CD-ROM), 166-168
 - velocidade linear constante (*Constant linear velocity* - CLV), 167
- Compactação, 227
- Comparação entre memória e circuitos lógicos, 30-31
- Compatíveis, 19
- Complemento a dois, 251-253, 255-267
 - algoritmo de restauração de divisão, 265-267
 - aritmética, 255-258
 - multiplicação, 260-265
 - operação, 255
 - representação, 251-253
 - representação geométrica, 257
- Componente discreto, 21-22
- Computação paramétrica, 536
- Computação vetorial, 541-551
 - conjunto de instruções da ALU, 550-551
 - encadeamento, 545-546
 - IBM 3090, facilidade vetorial, 546-551
 - instruções compostas, 550
 - organização registrador-a-registrador, 548-550
 - processamento de vetor, 544
 - processamento paralelo, 543-544
 - processamento paralelo matricial, 546
 - técnicas de pipeline, 543-546
- Computadores, 1-50
 - ARM (*Advanced RISC machine*), 1-2, 35-38
 - arquitetura, 1-11
 - circuitos integrados, 21-26
 - componentes, 9-11
 - dados, 8-9
 - desempenho, 29-45
 - memória semicondutora, 27
 - estrutura, 7, 9-11
 - evolução, 12-29, 34-38
 - função, 7-9
 - instruções, 13
 - introdução, 6-11
 - mapa para leitores e professores, 2-4
 - microprocessadores, 27-29
 - newsgroups USENET, 4
 - organização, 1-11
 - porque estudar, 2-3
 - projetando, 29-34
 - recursos da Internet e Web, 3-4
 - sistema Intel x86, 2, 34-35
 - transistores, 19-21
 - válvulas, 12-19
- Computadores comerciais, 18-19
- Computadores multicore, 513, 559-580
 - ARM11 MPCore, 571-575
 - aumento do tempo de sobrecargas, 563-564
 - aumento em paralelismo, 560-562
 - chips multiprocessador, 559
 - consumo de energia, 562-563
 - desempenho de software, 563-567
 - desempenho do hardware, 559-563
 - dimensionamento de banco de dados (*database scaling*), 564-566
 - granularidade, 566
 - Intel Core Duo, 568-570
 - Intel Core i7, 570-571
 - Intel x86, organização, 568-571
 - organização, 567-568
- Comunicação de dispositivo, módulos de E/S, 179
- Comutador, 203
- Comutador, InfiniBand, 203
- Conceito de família, 395
- Conceito de programa armazenado, conceito, 13
- Conflito de recurso, paralelismo, 434
- Conjuntos de instruções, 40, 247-248, 286-354, 414-416, 421-423, 550-551

- Advanced RISC Machine (ARM)*, 296, 313-315, 337-339, 348-349
- arquitetura, 40
- CPU, funções, 247-248, 286-354
- formatos de instrução, 248, 339-351
- IBM 3090, ALU de facilidade vetorial, 550-551
- instruções de máquina, 286-321
- linguagem de montagem, 350-351
- microprocessador MIPS R4000, 414-416
- modos de endereçamento, 247, 329-339
- operações (*opcode*), 247, 287, 297-315
- operandos, 247, 287-288, 292-293
- ordens de byte *endian*, 325-328
- pilhas, 321-325 constantes imediatas, ARM, 348
- projeto, 291-292
- RISC, 414-416, 421-423
- SPARC, 421-423
- tipos de dados, 294-296
- tipos de dados do x86, 294-295, 307-313, 335-337, 346-348
- Consumo de energia, 562-563
- Contador de programa (program counter – PC)*, 15, 56, 359
- Controlador Distribuído de Interrupções (*distributed interrupt controller – DIC*), 571-573
- Controlador programável avançado de interrupções (*Advanced Programmable Interrupt Controller – APIC*), 569
- Controladores, 73-74, 181, 188, 189, 193-195
- árbitro de barramento, 72-73
- canais, 197-198
- E/S, 181, 188, 193-196
- Intel 8237A, DMA, 193-196
- Intel 82C59A, interrupção, 188
- Controle, 179, 299, 303-307, 480-481, 484-487, 505-506
- implementação de pilha, 306-307
- instruções de chamada de procedimento, 305-307
- instruções de desvio, 303-304
- instruções de salto, 304-305
- lógica de, 178
- memória, 481-482
- microinstruções, 480-482, 486
- módulos de E/S, 179
- operações (*opcode*), 299, 303-307
- operações do sistema, 299, 303
- palavra, 480-481
- TI 8800 SBD, campos do microsequeenciador, 505-506
- transferência de, 297-300, 303-307
- Wilkes, controle microprogramado, 484-487
- Controle de acesso, 242-243
- Controle residual, 491
- Convertendo entre diferentes tamanhos em bits, 253-255
- Core, magnetização e, 27
- Correção de erro, 136-140
- bits de paridade, 137
- código de Hamming, 137
- erro não permanente, 136
- erro permanente, 136
- funções do código, 136
- memória interna, 136-140
- memória semicondutora, 136-140
- palavra síndrome, 138
- SEC (correção de único erro), código, 140
- Correção de único erro (SEC) Single Error-Correcting*, código, 140
- Cortex-A8, processador, 446-452
- arquitetura, 446-447
- instruções SIMD, 452
- paralelismo em nível de instrução, 446-452
- pipeline SIMD e de ponto flutuante, 452
- projeto superescalar, 446-452
- unidade de busca de instrução, 447-449
- unidade de decodificação de instrução, 449-450
- unidade de execução de inteiros, 450-452
- CPI (ciclos por instrução), 39-41
- CPSR, ARM, 387-389
- D**
- Dados, 8-9, 17, 18, 26, 178, 289, 292-295
- ARM, tipos de, 296
- armazenamento, 8
- canais, 26
- controle, 8
- E/S, 178
- instruções de máquina, tipos de, 289, 292-295
- instruções de transferência, 17, 18
- Intel x86, tipos de, 294-295
- movimentação, 8
- operandos, 292-293
- processamento, 8
- Daisy chain*, 188
- Decodificador, 484
- Dedicação física, 72
- Densidade de potência, 32
- Dependência de dados (fluxo) verdadeira, paralelismo, 432-434
- Dependência de saída, paralelismo, 436-437
- Dependências procedurais, paralelismo, 433
- Desempenho de disco espelhado (RAID nível 1), 158-159, 161-162
- Desempenho de disco não redundante (RAID nível 0), 157-161
- Desempenho de disco redundante via código de Hamming (RAID nível 2), 158-159, 162
- Desempenho de *striped disks* (RAID nível 0), 159-161
- Desempenho do computador, 38-57
- ARM, 35-38
- balanço do, 30-31
- velocidade do microprocessador, 29-30
- lei de Amdahl, 44-45
- organização do chip, 31-34
- programas de *benchmark*, 41-44
- projetando visando, 29-34
- sistema Intel x86, 34-35
- sistemas embarcados, 35-36
- velocidade de *clock*, 39-41
- Desempenho do disco com paridade distribuída em nível de bloco (RAID nível 5), 158-160, 164
- Desempenho do disco com paridade em nível de bloco (RAID nível 4), 158-160, 163-164
- Desempenho do disco com paridade intercalada por bit (RAID nível 3), 158-160, 162-163
- Desempenho do disco com redundância dual (RAID nível 6), 158-160
- Deslocamento aritmético, 263, 302
- Deslocamento (*shift*) lógico, 301-302
- Desvio atrasado, pipeline, 377, 41-412
- Desvios, 17-18, 289, 303-304, 339, 366-367, 372-377, 411-412, 439-440
- atrasados, 377, 41-412
- buffer de laço para, 373
- busca antecipada do alvo do, 372
- fluxos múltiplos para, 373
- hazard* de controle, 372
- instruções, 289, 303-304, 339
- instruções condicionais, 16-17, 303-304, 366-367
- instruções incondicionais, 16-17, 303-304
- instruções RISC, 411-412
- pipeline, 366-367, 372-377, 411-412
- previsão, 374-377, 439-440
- processadores superescalares, 440
- Deteção de erro, módulos de E/S, 180
- Diagramas de estado, operações de instrução, 59, 64, 363
- Diferença de desempenho entre lógica e memória, 30-31
- Diferença semântica, 397
- Digital Equipment Corporation (DEC)*, computadores da série PDP, 20, 25-26, 341-344
- Digital Versatile Disk (DVD)*, 166, 168-169
- Disco com cabeça fixa, 153
- Disco com cabeça móvel, 153
- Disco de dupla face, 153
- Disco magnético, 149-157
- atraso rotacional (latência), 155-156
- cabeça, 150, 153-154
- cilindro, 153
- contato (disquete), 153
- deteção de posição rotacional (*rotational positional sensing – RPS*), 155
- face única e dupla, 153
- formatação de dados, 151-152
- formato Winchester, 153, 154
- gravação em múltiplas zonas, 152
- leitura magnética, 150
- mecanismos de gravação, 150
- múltiplas placas, 153
- organização sequencial, 156-157
- parâmetros, 155-157
- tempo de busca (*seek time*), 155

tempo de transferência, 155-156
 trilhas, 151, 154
 velocidade angular constante (*constant angular velocity* – CAV), 151
 Disco não removível, 153
 Disco removível, 153
 Disco rígido, 153
 Discos. *Ver* discos magnéticos; sistemas de memória óptica (despachante), 220
 Discos de única face, 153
 Discos ópticos de alta definição (HD DVD), 169
 Dispositivos em estado sólido, 19
 Dispositivos externos (periféricos), E/S, 177-178
 Disquete (mecanismo de contato), discos magnéticos, 153
 Divisão, 265-267, 275-279
 algoritmo de restauração de complemento de dois, 266-267
 números na representação com (ou em) ponto flutuante, 275-279
 resto parcial, 265-266
Double-Data-Rate SDRAM (DDR-SDRAM), 144
 DRAM Rambus (RDRAM), 143
 DRAM síncrona (SDRAM), 141-143
 Drive, processador Pentium 4, 442-445
Dynamic random-access memory (DRAM, 29-30, 129-131, 132-133, 140-145
 cache DRAM, 145
 DDR-SDRAM, 144
 DRAM Rambus, 143
 DRAM síncrona, 141-143
 lógica do chip, 132-133
 memória principal interna, 129-131
 processador de alto desempenho, 140-145

E

E/S controlada por interrupção, 176, 181-183, 184-191
 desvantagens, 191-192
 E/S programada, 181-183, 188-191
 execução, 176-177, 181-183
 Intel 82C55A, interface de periférico programável, 188-191
 Intel 82C59A, controlador de interrupção, 188
 linhas de interrupção múltiplas, 187
 processamento de interrupção, 184-186
 projeto e implementação, 186-188
 técnica de arbitragem de barramento, 188
 técnica de consulta por software, 187-188
 técnica de *daisy chain*, 188
 E/S independente, 183
 E/S mapeada na memória, 183
 E/S programada, 176, 181-184, 188-191
 comandos, 182-183
 desvantagens, 191
 E/S controlada por interrupção, 184-186
 execução, 176, 181-184
 independente, 183
 instruções, 183-184
 Intel 82C55A, interface de periférico

programável, 188-191
 mapeada na memória, 183
 E/S via conjunto teclado/monitor, 179
 E/S, canais, 181, 197-198
 E/S, módulos, 67-68, 176-177, 179-181, 196-197
 buffering de dados, 180
 comunicação com o dispositivo, 180
 comunicação do processador, 68, 179-180
 detecção de erro, 180
 estrutura de interconexão, 67-68
 estrutura, 180-181
 evolução, 196-197
 função, 66-67, 179-180
 funções do computador, 66
 interfaces de entrada/saída, 176-177
 requisitos de controle e tempo, 179-180
 EFLAGS, registrador, processadores Intel x86, 379-380
Electrically erasable programmable read-only memory (memória somente de leitura programável e apagável eletricamente – EEPROM), 129, 132
 Elemento de controle de sistema (*system control element* – SCE), 521
 Encadeamento, 545-546
 Endereçamento carregar/armazenar (*load/store*), ARM, 337
 Endereçamento de carga/armazenamento múltiplo (*load/store*), ARM, 339
 Endereçamento de carga/armazenar (*load/store*), ARM, 337
 Endereçamento de instruções de processamento de dados, ARM, 337-339
 Endereçamento múltiplo carregar/armazenar (*load/store*), ARM, 338
 Endereçamento por registrador base, 333
 Endereço de base, 228
 Endereço físico, 228, 229
 Endereço lógico, 228, 229
 Endereço relativo, 228-229, 333
 Endereços, 98-100, 227-229, 234, 239-240, 289-291
 acumulador (AC), 290
 base, 228
 campos, 235
 espaços, 234
 físicos, 228, 229
 gerenciamento de memória de E/S, 227-228, 234, 239-240
 instruções de máquina, 289-291
 lógicos, 228, 229
 memória cache, 98-100
 memória virtual, 235, 239-240
 número de, 289-291
 particionando, 227
 Pentium II, mecanismos de tradução, 234-238
 relativos, 229
 tabelas de página, 228-229, 239-240
 tradução ARM, 239-240
Endian, ordens de byte, 325-328

ENIAC (*electronic numerical integrator and computer*), 12-13
 Entrada/saída (E/S), 10, 52, 55-56, 67-68, 160-161, 176-209, 299, 303
 alta capacidade de transferência de dados, 160-161
 alta taxa de solicitação, 161
 barramento serial *FireWire*, 199-202
 canais, 181, 197-198
 conjunto teclado/monitor, 179
 controlada por interrupção, 176, 184-191
 controladores, 181, 188, 193-196
 desempenho do RAID 0 para, 160-161
 dispositivos de dados periféricos, 8
 dispositivos periféricos (externos), 177-179
 DMA, 68, 176, 191-196
 estrutura de interconexão, 67-68
 função, 196-197
 funções componentes, 21
 InfiniBand, 202-205
 Intel 8237A, controlador de DMA, 193-196
 Intel 82C55A, interface periférica programável, 188-191
 Intel 82C59A, controlador de interrupção, 188
 interfaces, 177, 188-191, 198-205
 interfaces multiponto, 199
 interfaces ponto-a-ponto, 199
 módulos, 67-68, 176-177, 179-181, 196-197
 movimento de dados, 8
 operações (*opcode*), 299, 303
 programada, 176, 181-184, 188-191
 registrador de buffer E/S (I/OBR), 55-56
 registrador de endereço E/S (I/OAR), 55-56
 sistemas de computador, 52, 55-56, 176-209
 unidade de disco, 179
 técnicas de execução, 176, 181-183
 EPROM (memória somente de leitura programável e apagável), 129, 132, 134
 empacotamento de chip, 134
 memória principal interna, 132, 134
 Erro não permanente, 136
 Escalonamento, 210, 214, 219-224, 446, 528
 curto prazo, 220-224
 eficiência, 213-214
 estado de um processo, 221
 filas, 223-224
 longo prazo, 219-220
 médio prazo, 220
 micro-operações (micro-ops), 442
 multithreading, 528
 processo de interrupção, 223
 processo, 219-221, 528
 sistema operacional (SO), função, 210, 219-224
 técnicas, 220-222
 Escalonamento a curto prazo, 220
 Escalonamento a longo prazo, 219-220
 Escalonamento de banco de dados, 564-567
 Escalonamento de médio prazo, 220

Estado de processo, 220-221
 Estrutura de computador de alto nível, 10-11, 51-88
 funções, 7-9, 51-66
 ciclo de instrução, 15-18, 56-59, 60-66
 interconexões, 10-11, 66-83
 diagramas de temporização, 87-88
 ciclo de busca, 16, 56-59
 ciclo de execução, 17, 56-59
 Exceções, interrupções e, 383, 389
 Execução, 39-41, 176, 181-182, 222-224, 396-400, 435-437, 440, 445-464, 468, 492-501, 528
 codificação, 495-496
 fora de ordem, 436-437, 445-446
 IBM 3033, processador, 499-501
 microinstruções, 492-501
 multithreading, 528
 processador LSI-11, 496-499
 processo, 222-223, 528
 programas superescalares, 440
 RISC, instruções de máquina, 396-400
 taxa de instruções, 39-41
 taxonomia, 492-494
 técnicas de E/S, 176, 181-182
 unidade de controle, 468, 492-501
 Execução fora de ordem, 445-446

F

Falha permanente, 136
 Falta de página (*page fault*), 230
 FIFO (*first-in first-out* - primeiro a entrar, primeiro a sair), algoritmo, 109-110
 Filas intermediárias, 225-226
 Filas, 223, 446
 E/S, 211-212
 escalonamento de processador, 224
 fita de longo prazo, 223
 intermediária, 225
 micro-operações (micro-ops), 466
 troca de processos na memória, 224-225
Fire Wire, barramento serial, 199-202
 camada de enlace, 200-202
 camada de transação, 200-201
 camada física, 200-201
 configurações, 199-200
 cycle master, 250
Firmware (ou *microprograma*), 480
Fita linear aberta (*linear tape-open* - LTO), sistema de cartucho, 171
 Fita magnética, 169-171
Flags de status, 310
 Fluxo de dados, ciclos de instruções, 362-364
 Formatação de dados, 151
 Formatos de instrução, 248, 339-351, 408-409, 416-417, 423-424
 alocação de bit, 340-343
 ARM, 348-349
 Frames, memória de E/S, 228-229
 linguagem de montagem, 350-351
 microprocessador MIPS R4000, 507

PDP-10, projeto, 342-343
 PDP-11, projeto, 343-344
 PDP-8, projeto, 341-342
 RISC, 408-409, 417, 423-424
 SPARC, 423-424
 tamanho variável, 343
 tamanho, 340
 VAX, design, 344-346
 x86, 346-348
 Formatos de instrução de tamanho variável, 343-346
Front end, processador Pentium 4, 442-445
 Funções, 7-9, 17, 19, 53-66, 83-86, 179-180, 196-197
 arquitetura de von Neuman, 54-56
 canais de E/S, 197-198
 ciclo de busca, 16, 56-59
 ciclo de execução, 17, 56-59
 ciclo de instrução, 16-17, 56-59, 60-66
 componentes de software, 54-55
 componentes, 19, 53-66
 entrada/saída (E/S), 66, 179-180, 196-197
 interrupções, 54-66
 módulos de E/S, 66, 179-180, 196-197
 operação do computador IAS, 16-17
 operação do computador, 7-9
 programas *hardwired*, 54
 Funções de mapeamento, 100-109
 associativas em conjunto, 104-109
 associativas, 104-106
 direto, 101-104
 memória cache, 100-109

G

Gerenciador de recursos, 50, 212-213, 218-219
 Gerenciamento de falha, *clusters*, 536
 Gerenciamento de memória, 210, 219, 224-243
 Advanced RISC machine (ARM), 238-243
 compactação, 227
 controle de acesso, 242-243
 endereços, 227-228, 234, 239-240
 entrada/saída (E/S), 211, 219, 224-243
 formatos, 236, 240-241
 Intel Pentium II, processador, 234-238
 Intel x86, instruções de máquina, 309
 memória virtual, 229-231, 239-240
 multiprogramação, 219, 224
 paginação, 228-230, 235-238
 parâmetros, 237, 242
 particionamento, 226-228
 segmentação, 233-234, 234-235
 sistema operacional (SO), 210, 219, 224-243
 swapping, 224-226
 Translation lookaside buffer (TLB), 232-233, 238-239
 Gravação em múltiplas zonas, 152
 Gravação em serpentina, 170-171
 Gravação paralela, 170
 Gravação serial, 170

H

Hardware, 524-525, 559-563
 aumento de paralelismo, 560-562
 consumo de energia, 562-563
 desempenho de MPCore (computadores multicore), 559-563
 soluções de coerência de cache, 524-525
Hash, funções, 231
Hazards de controle, pipeline, 372
Hazards de dados, pipeline, 372
Hazards de recurso, pipeline, 70-371
Host Channel Adapter (HCA), 203

I

IAS, computador, 15-18
 IBM. *Ver* International Business Machines (IBM)
 IEEE. *Ver* Institute of Electrical and Electronics Engineers (IEEE)
 Imagem de sistema único, 536
 Indexação, 334
 InfiniBand, 202-205
 InfiniBand, pistas virtuais, 204-205
 infinito, interpretação do padrão IEEE, 279
Institute of Electrical and Electronics Engineers (IEEE), 2-3, 271-272, 277-280
 interpretação de infinito, 279
 Joint Task Force, publicações, 2-3
 NaN, padrões, 279
 padrões de notação de números em ponto flutuante, 271-272, 277-280
 padrões numéricos desnormalizados, 279
 técnicas de arredondamento, 27-279
 Instruções aritméticas, 17-18, 289
 Instruções *call/return*, 309-310
 Instruções compostas, facilidade de vetor do IBM 3090, 550
 Instruções de construir ou retirar, 440
 Instruções de desvio condicional, 17-18, 303-304, 367-368
 Instruções de desvio incondicionais, 17-18, 303-304
 Instruções de máquina, 286-321, 396-400
 aritméticas, 289
 chamada de procedimento, 305-307, 309, 399-400
 conjunto de instruções reduzido (*reduced instruction set computers* - RISC), 396-400, 409
 desvio-(*branches*), 289, 303-304
 elementos, 287-288
 endereços, 289-291
 execução RISC, 396-400
 Intel x86, 294-295, 307-313
 linguagens de alto nível (HLL), 396-399
 lógicas (Booleanas), 289
 memória, 289
 operações (*opcode*), 287, 297-315, 397-398
 operandos, 287-288, 292-293, 398-399
 projeto do conjunto de instruções, 291-292
 representação simbólica, 288-289
 teste, 289

- tipos de dados, 287-287, 292
 - tipos de dados do ARM, 296, 314, 315
 - Instruções. *Ver* instruções de máquina, micro-operações
 - Instruções de memória, 289
 - Instruções de modificação de endereço, 18
 - Instruções de salto, 304-305
 - Instruções de teste, 289
 - Instruções lógicas (Booleanas), 289
 - Instruções privilegiadas, 216
 - Instruction register (IR - registrador de instrução), 15, 56, 359-360
 - Integração de pequena escala (SSI) Small-Scale Integration*, 23
 - Integração em larga escala (*large-scale integration* - LSI), 26
 - Inteiros, 250-267, 446, 450-452
 - adição, 256-258
 - aritmética, 255-267
 - complemento a dois, 251-253, 255-267
 - conversão entre tamanhos de bit, 253-255
 - divisão, 265-267
 - multiplicação, 258-265
 - multiplicação não sinalizada, 259-260
 - negação, 255-256
 - overflow*, 256-258
 - ponto fixo, 255
 - representação, 250-255
 - sinal-magnitude, 251
 - subtração, 256-258
 - unidade de execução do processado Cortex-A8, 450-452
 - unidade de execução do processador Pentium 4, 446
 - Unidade Lógica e Aritmética (ALU), dados, 250-267
 - Intel x86, sistema, 1-2, 34-35, 188-191, 193-196, 234-238, 294-295, 307-313, 335-337, 346-348, 360-361, 377-385, 441-446, 472-475, 532, 568-571
 - 80386, registradores de microprocessador, 360-361
 - 80486, pipeline de informação, 377-378
 - 8086, registradores de microprocessador, 360-361
 - 8237A, controlador de DMA, 193-196
 - 82C55A, interface de periférico programável, 188-191
 - 82C59A, controlador de interrupção, 188
 - Acesso Direto à Memória (*Direct memory access* - DMA) e, 193-196
 - códigos de condição, 310
 - Core Duo, 568-570
 - Core i7, 570-571
 - CPU, conjuntos de instruções, 294-295, 307-313, 335-337, 346-348
 - E/S controlada por interrupção, 188-191
 - E/S programável, 188-191
 - evolução, 34-35
 - flags* de status, 310
 - formato de instrução, 346-348
 - gerenciamento de memória, 234-238
 - instruções de *call/return*, 309-310
 - chip multiprocessador, 531-532
 - instruções de gerenciamento de memória, 310
 - MMX (tarefa de multimídia), instruções, 310-313
 - instruções de máquina, 294-295, 307-313
 - MMX, registradores, 382
 - modo de endereçamento, 335-337
 - memória cache, 113-115
 - operação de temporização, 474-475
 - operações (*opcode*), 307-313
 - organização de computador multicore, 568-571
 - organização do processador, 378-385
 - organização do registrador, 378-382
 - paralelismo em nível de instrução, 441-446
 - Pentium 4, processador, 113-115, 441-446, 532
 - Pentium II, processador, 234-238
 - processador 8085, unidade de controle, 472-475
 - processamento de interrupção, 382-385
 - projeto de processador superescalar, 441-446
 - registrador de controle, 380-382
 - registrador EFLAGS, 379-380
 - SIMD, instruções, 310-313
 - sinais externos, 473
 - tipos de dados, 294-295
 - Interconexão de barramento do sistema, 10, 68-79
 - elementos de projeto, 72-76
 - estrutura PCI, 76-79
 - estrutura, 68-70
 - hierarquia de barramento múltiplo, 70-72
 - largura do barramento de dados, 68, 75
 - Interconexão do sistema (*bus*), 10, 68, 356-357
 - linhas de controle, 69-70
 - linhas de dados, 68
 - linhas de endereço, 68-69
 - método de arbitração, 72-73
 - temporização, 73-75
 - tipo de transferência de dados, 75-76
 - Interconexões, 10-11, 51, 67-86, 522
 - barramento, 10, 68-76
 - componente periférico (PCI), 76-83
 - comutadas, SMP, 522
 - estrutura do computador, 9-11, 51
 - trocadas de dados, 68
 - módulos de E/S, 67-68
 - módulos de memória, 67
 - sinais do processador, 68
 - Interface serial de E/S, 198-199
 - Interface usuário/computador, SO, 211-212
 - Interfaces, 176-177, 188-191, 198-205
 - barramento serial FireWire, 199-205
 - E/S externa, 198-205
 - E/S paralela, 198-199
 - E/S serial, 198-199
 - entrada/saída (E/S), 176-177, 188-191, 198-205
 - InfiniBand, 202-205
 - Intel 82C55A, periférico programável, 188-191
 - módulos de E/S, 176-177
 - multiponto, 199
 - ponto-a-ponto, 199
 - Interfaces de E/S paralela, 198
 - Interfaces multiponto, 199
 - Interfaces ponto-a-ponto, 199
 - International Business Machines (IBM), 19-21, 23-25, 489-491, 499-501, 521-523, 532, 546-552
 - 3090, facilidade vetorial, 546-552
 - 700/7000, computadores seriais, 19-21
 - computador da série 360, 23-25
 - conjunto de instruções da ALU, 546-552
 - execução de instrução composta, 550
 - mainframes z990 SMP, 521-523
 - organização registrador-a-registrador, 548-550
 - Power5, chip de multiprocessamento, 532
 - processador 3033, microinstruções, 489-491, 499-501
 - sequenciação de geração de endereço, 489-491
 - International Reference Alphabet (IRA)*, 179
 - Interrupções, 59-66, 184-188, 189, 216, 223, 382-385, 389, 569, 571-573
 - Advanced programmable interrupt controller (APIC - controlador programável avançado de interrupções), 569
 - ARM 11 MPCore, 571-573
 - ARM, 389
 - ciclo de instrução, 59-66
 - desabilitadas, 64
 - DIC, 572-573
 - exceções, 383, 387
 - fluxo de controle do programa, 59-60
 - Intel 82C59A, modos, 188
 - Intel x86, processamento, 383-385, 568-569
 - modo de máscara especial, 188
 - modo de rotação, 188
 - modo totalmente aninhado, 188
 - MPCore (computadores multicore), 569, 571-573
 - múltiplas, 63-66, 187-188
 - processamento, 184-186
 - processo de escalonamento, 222-223
 - sinal de requisição, 60
 - SO (sistema operacional), 216
 - tabelas de vetor, 383-384, 389
 - tratamento, 62, 384-385, 572-573
 - vetorados, 187
 - Intervalos e arbitração de imparcial, 201
 - ISR (*Interrupt Service Routine*), 65-66
- ## J
- Janela de instrução, 436
 - Janelas, aumento de tamanho do banco de

registradores usando, 400-402, 420-421
 Job, sistema operacional (SO), 214
 Job control language (JCL), 215

K-L

Kernel (núcleo), 213
 Laço desenrolado, pipeline, 412-413
 Lacunas entre registros, 170
 Lacunas, discos magnéticos, 151
 Lei de Amdahl, 44-45, 563-564
 Lei de Moore, 23-24
 Leitura atrasada, pipeline, 412
 Leitura com acerto/falha, 526-527
 Leitura com intenção de modificar (RWITM), 527
 Leitura magnética e mecanismo de gravação, , 150
 LFU (*Least-Frequently Used*), algoritmo, 109-110
 Linguagem de alto nível (HLL - high-level language), 122, 396-400
 características de desempenho, 152-153
 chamadas de procedimento, 399
 conjunto de instruções reduzido (RISC - *reduced instruction set computers*), 396-400
 diferença semântica, 397
 operações, 397-398
 operandos, 398-399
 Linguagem de microprogramação, 480
 Linguagem de montagem, 350-351
 Linhas, memória cache, 87-88, 89, 95-96, 111, 113
 Linhas de controle, 69
 Linhas de dados, 68-69
 Linhas de endereço, 68-69
 Linhas de sinal, PCI, 78
 Linhas migratórias, 574-575
 Links, *InfiniBand*, 203-204
 Little endian, ordenação, 325-328
 Livros, mainframes SMP, 521-523
 Localidade de referência, 93, 122-123
 Localidade espacial, 123
 Localidade temporal, 123
 Lógica de gerenciamento de energia, 569-570
 LRU (*Least-Recently Used*), algoritmo, 109-110, 230
 LSI (Integração em grande escala), 26
 LSI -11, processador, 491, 496-499
 execução, 496-499
 formato de microinstrução, 497-499
 microinstruções, 491, 496-499
 sequenciamento, 491
 unidade de controle, organização, 496
 LTO (fita linear aberta), sistema, 171

M

Mantissa, 268
 Mapeamento associativo em conjunto, 106-109
 Mapeamento associativo, 104-106
 Mapeamento direto, 101-104
 Mecanismos de gravação, discos magnéticos, 149-150
 Média aritmética, 42

Média geométrica, 43
 Média harmônica, 42-43
 Memória cache de disco, 122
 Memória cache em dois níveis, 121-126, 523
 Memória cache multinível, 111-113
 Memória cache separada, 113
 Memória cache unificada, 113
 Memória cache, 51, 89-127, 396, 402-404, 522-523, 571, 573-575
 Memória cache, 98-99, 122
 algoritmos de substituição, 109-110
 Computação de alto desempenho (HPC), 98
 desempenho de disco, 122
 desempenho de dois níveis, 98-99, 122
 desenvolvimento, 395
 dois níveis, 121-127, 522-523
 elementos de design, 98-113
 endereço físico na memória principal, 98-99, 121-122
 endereços, 98-99
 estrutura, 95-98
 função de mapeamento, 100-104
 MPCore (computadores multicore), 571, 573-575
 grande banco de registradores *versus*, 402-404
 linhas, 95-97, 111
 localidade de referência, 122-123
 multinível, 111-112
 nível de hierarquia, 92-94
 Linguagem de alto nível (HLL), operações, 122-123
 organização ARM, 115-116, 571, 573-575
 organização do Pentium 4, 113-115
 política de escrita, 110-111
 separadas, 113
 SMP compartilhada, 522-523
 tags, 95
 tamanho, 99-100, 111
 unidade de controle de monitoramento (SCU), 571, 573-575
 unidade de gerenciamento da (MMU), 98-99
 unificadas, 113
 Melhoria da velocidade, 368-370, 563-564
 Memória de acesso aleatório (RAM), 129-131
 Memória externa, 52, 149-175
 dispositivos de acesso direto, 170
 discos magnéticos, 149-157
 dispositivos de acesso sequencial, 170
 fita magnética, 169-171
 RAID (*Redundant array of independent disks*), 149, 157-164, 165, 166
 sistemas ópticos, 164-169
 Memória flash, 129, 132
 Memória intercalada (*interleaved memory*), 135-136
 Memória interna, 52, 128-148
 chips, 132-135
 correção de erro, 136-140
 desempenho de alto nível, 140-145
 DRAM, 129-131, 132-133, 140-145

EEPROM, 129, 132
 EPROM, 129, 132, 134
 intercalada, 135-136
 memória flash, 129, 132
 principal (célula), 128-136
 PROM, 129, 131
 RAM, 129-130
 ROM, 129, 131-132
 semicondutores, 128-148
 SRAM, 131
 Memória principal, 9, 55, 98, 99, 122, 128-131, 213-216
 cache (física), 98-99, 122
 componente do computador, 9-10, 55
 gerenciador de recurso do SO, 212-213
 interna (célula), 128-136
 kernel (núcleo), 213
 Memória real, 245
 Memória secundária (auxiliar), 94
 Memória semicondutora, 27, 128-148. *Ver também* memória interna
 memória somente de leitura (ROM), 129, 131-132
 Memória virtual, 229-231, 239-240
 campos de endereço do Pentium II, 234
 estrutura de tabela de página invertida, 230-231
 paginação por demanda, 229-230
 gerenciamento de memória, 229-231, 239-240
 substituição de página, 230
 tradução de endereço no ARM, 239-240
 MESI (*Modified, Exclusive, Shared, Invalid*), protocolo, 525-527
 mestre de ciclo, 202
 Método de acesso, 91
 Método de *cluster* de secundário passivo (*passive standby*), 534-535
 Método de *clustering* secundário ativo, 535
 métodos de *clustering* de servidor, 535
 Métrica de taxa, 43
 Métrica de velocidade, 42
 Microeletrônica, desenvolvimento, 22-23
 Microinstruções, 480-482, 487-510
 codificação, 495-496
 execução, 492-501
 formatos, 497-499, 502-504
 horizontal, 481, 494
 IBM 3033, processador, 489-491, 499-501
 LSI-11, processador, 491, 496-499
 memória de controle, 481
 sequenciamento, 487-491, 504-506
 taxonomia, 492-494
 TI 8800 *Software Development Board* (SDB), 501-509
 verticais, 484, 494
 Wilkes, exemplo de, 486
 Microinstruções do LSI-11, 491, 496-497
 execução, 496-499
 formato da microinstrução, 497-499
 microinstruções, 491, 496-499
 sequenciamento, 491

unidade de controle, organização, 496-497

Micro-operações (micro-ops), 441-442, 445-446, 462-466

alocação, 445

ciclo de busca, 463-464

ciclo de execução, 465-466

ciclo de instrução, 466

ciclo de interrupção, 465

ciclo indireto, 464

filas de, 446

escalonamento de despacho, 446

geração *front end*, 442-443

Pentium 4, processador, 441-442, 445-446

processadores superescalares, 441-442

unidade de controle, 462-466

Microprocessadores, 27-29, 29-30, 360-361

desenvolvimento, 27-29

organizações de registrador, 360-361

registradores Intel 80386, 360-361

registradores Intel 8086, 360-361

registradores Motorola MC68000, 360-361

velocidade (desempenho), 29-30

Milhões de instruções por segundo (MIPS), taxa, 40

Milhões de operações de ponto flutuante por segundo (MFLOPS), taxa, 41

MIPS R4000, microprocessador, 413-419

conjunto de instruções, 414-416

formato de instrução, 416

pipelining de instruções, 416-419

MMX (tarefas de multimídia), processadores Intel

x86, 310-313, 382

instruções, 310-313

registradores, 382

Mnemônicos, 288

Modo de interrupção totalmente aninhado, 189

Modos de endereçamento, 248, 329-339, 407-408

Advanced RISC Machine (ARM), 337-339

conjuntos de instruções da CPU, 248, 329-339

deslocamento, 331-332, 333-334

direto, 331-332

imediato, 331

indireto, 332

Intel x86, 335-337

pilha, 330-332, 334-335

módulo de E/S, 179-180

registrador indireto, 331, 333

registrador, 332-333

simplicidade do RISC, 407-409

Módulo de interrupção rotativo, 188-189

Módulos de memória, 67

Monitor (SO em lote simples), 214-216

Monitor residente, 214

Motorola MC68000, registradores de

microprocessador, 360-361

Mudança de fase, 168

Múltiplas linhas de interrupção, E/S, 187

Múltiplas placas, discos magnéticos, 153

Multiple instruction, multiple data (MIMD), 515-517

Multiple instruction, single data (MISD), 515-517

Multiplexação de tempo, 72

Multiplexador, 21

Multiplexador de bloco, 198

Multiplexador de byte, 197-198

Multiplicação, 258-265, 275-279

algoritmo de Booth, 263-265

complemento de dois, 260-265

inteiros sem sinal, 259-260

números de ponto flutuante, 275-279

Múltiplo processamento paralelo, 546

Múltiplos fluxos, *pipelining*, 372-373

Multiprocessadores simétricos (SMP), 512, 514-523, 538

arquitetura paralela de processador, 515-516

caches L2 compartilhadas, 523

características do sistema, 517-518

clusters comparados com, 538

considerações sobre projeto de SO para

interconexões chaveadas, 522

multiprocessadores, 520-521

mainframe, 521-523

mainframes IBM z990, 521-523

organização, 518-519

Multitarefa, sistemas operacionais (SO), 216

Multithread, 514, 528-532

chip multiprocessadores, 514, 529

explícito, 528-532

implícito, 528-529

processamento paralelo, 514, 528-532

processo, 528-529

troca, 528

thread, 528

Multithreading em bloco, 529-531

Multithreading intercalado, 529-531

Multithreading simultâneo (SMT), 529-531

N

NaNs, padrões do IEEE, 279

Negação, inteiros, 255-256

Nonuniform memory access (NUMA), 512, 514-516, 539-541

com coerência de cache (CC-NUMA), 539-541

motivação, 539

organizações, 539-541

prós e contras, 541

sistemas de processadores paralelos, 514-516

Uniform memory access (UMA), 539

notação de infix, 323

Notação de números na representação com (ou em) ponto fixo. *Ver* partições inteiras de tamanho fixo, 226-227

Notação polonesa invertida, 323-324

Notação pós-fix, 323

Números desnormalizados, padrões IEEE, 279-280

Números normalizados, 269

O

Offset de endereçamento, ARM, 337

Opcode. *Ver* operações (*opcode*)

Operação de pilha POP, 321

Operações (*opcode*), 15, 18, 287, 297-315, 397-398

Advanced RISC Machine (ARM), 313-315

aritméticas, 297, 300

controle do sistema, 297, 303

conversão, 298, 302-303

entrada/saída (E/S), 298, 303

instrução de máquina, 287, 297-315

instruções de computador, 14, 18

Intel x86, 307-313

linguagem de programação de alto nível (*High-Level Language* – HLL), 397-398

lógica, 298, 300-302

Reduced Instruction Set Computers (RISC), 397

transferência de controle, 299, 303-307

transferência de dados, 297-300

Operações aritméticas (*opcode*), 297, 300

Operações de controle do sistema, 297-299, 303

Operações de conversão, 298, 302-303

Operações de transferência de controle, 297, 303-307

Operações lógicas (*opcode*), 298, 300-302

Operandos, 247, 287-288, 292-293, 398-399

caracteres, 293

dados lógicos, 293

instrução de máquina, 287-288

linguagem de alto nível (HLL), 398

números, 292-293

Reduced Instruction Set Computers (RISC), 399

representação decimal agrupada, 292-293

Operandos de dados de caractere, 293

Operandos de dados lógicos, 293

Operandos de dados numéricos, 292-293

Ordenação de bits, *endian*, 328

Ordenação de bytes, *endian*, 325-328

Organização paralela, 512-576

acesso não uniforme à memória (*nonuniform memory access* – NUMA), 513, 514-516, 538-541

chip de multiprocessamento, 514, 531-532

clusters, 514, 532-538

coerência de cache, 514, 523-525

computação vetorial, 541-551

computadores multicore, 513, 559-576

multiprocessadores simétricos (*symmetric multiprocessors* – SMP), 513, 514-523, 538

multithreading, 514, 528-532

organizações de múltiplos processadores, 515-516

processamento paralelo, 513-558

Organização registrador-para-registrador, 407-408

Organização sequencial, discos magnéticos, 156-157

Original equipment manufacturers (OEMs), 25

Ortogonalidade, 344

Otimização de registradores baseada em compiladores, 404-405

Overflow, 256-257, 270, 273

P

- Paginação por demanda, 229-230
- Paginação, 228-231, 235-238
 - alocação de frame, 228
 - demanda, 229-230
 - gerenciamento de memória, 228-231, 238-243
 - memória virtual, 229-231
 - processador Pentium II, 235-238
 - substituição de página, 230
 - tabela de página, 228-229, 230-231
- Palavra do estado do programa (*program status word* – PSW), 359
- Palavra syndrome, 137-138
- Palavras, 15, 90, 340
- Paralelismo de máquina, 434, 438-439
- Paralelismo, 248, 429-458, 536, 560-562
 - aplicações de *cluster*, 536-537
 - aumento de computador multicore, 559-562
 - de máquina, 434, 438-439
 - limitações, 432-434, 435-436
 - nível de instruções, 247, 429-458
 - política de emissão de instruções, 44-437
- Paralelismo em nível de instrução, 248, 429-457
 - antidependência, 436-437
 - ARM, processador Cortex-A8, 446-452
 - conflito de recurso, 433-434
 - dependência de dados verdadeira, 432-434
 - dependência de saída, 436
 - dependência procedural, 433
 - execução de programas superescalares, 440
 - implementação de programas superescalares, 441
 - Intel Pentium 4, processador, 441-446
 - paralelismo de máquina, 434, 438-439
 - paralelismo em nível de instrução, 432-434
 - política de emissão de instrução, 434-437
 - previsão de desvio, 439-440
 - processadores superescalares, 248, 396, 429-457
 - renomeação de registrador, 437-438, 446
- Parâmetros, discos magnéticos, 155-157
- Particionamento, gerenciamento de memória, 224-228
- Partições de tamanho variável, 226-227
- PCI. *Ver Peripheral Component Interconnection (PCI)*
- PDP-10, conjunto de instruções, 342-343
- PDP-11, projeto de instrução, 343-344
- PDP-8, projeto de instrução, 341-342
- Pentium 4, processador, 113-115, 441-446, 532
 - alocação, 445
 - busca na trace cache, 445
 - chip multiprocessadores, 531
 - drive, 443, 445
 - front end*, 442-443
 - lógica de execução fora de ordem, 445-446
 - micro-operações (micro-ops), 441-442, 445-446
 - organização, 113-115
 - paralelismo em nível de instrução, 441-446
 - ponteiro da próxima instrução da trace cache, 442-445
 - projeto superescalar, 441-446
 - renomeação de registradores, 446
 - unidade de execução de inteiros, 446
 - unidade de execução de ponto flutuante, 446
- Pentium II, processador, 234-238
 - campos de endereço virtual, 235
 - espaços de endereço, 234
 - formatos para gerenciamento de memória, 236
 - gerenciamento de memória de E/S, 234-238
 - paginação, 235-238
 - parâmetros de gerenciamento de memória, 237
 - segmentação, 234-235
- Peripheral Component Interconnection (PCI)*, 76-83
 - arbitração, 82-83
 - comandos, 79-80
 - configuração, 76-77
 - estrutura de interconexão de barramento, 76-79
 - linhas de sinal, 78
 - sinal de concessão (GNT), 82-83
 - sinal de requisição (REQ), 82-83
 - Special Interest Group (SIG)*, 76
 - transferências de dados, 80-82
- Pilhas, 290, 306, 307, 324, 329, 331, 334, 335, 505
 - avaliação de expressão, 323-324
 - frames, 307
 - implementação de chamada de procedimento, 306-307
 - implementação de processador, 322-323
 - instruções com zero endereço, 291
 - microsequenciador TI 8800 SBD, 619
 - modo de endereçamento, 329-331, 334-335
 - operações, 32-322
 - ponteiro (*stack pointer* – SP), 323
- Pipeline, 364-378, 396, 417-419, 452, 543-546
 - bolha, 370
 - buffer* de laço, 373
 - busca antecipada de instrução (busca sobreposta), 365, 372-373
 - ciclo de tempo, 369
 - desempenho, 368-370
 - instrução de processador, 364-378
 - desenvolvimento, 395-396
 - desvio atrasado, 377, 411-412
 - desvios, 372-377
 - estratégia, 364-368
 - fator de melhoria da velocidade, 368-369
 - hazards, 370-372
 - instruções de ponto flutuante, 452, 545-546
 - instruções RISC, 410-413, 416-419
 - Intel 80486, processador, 377-378
 - laço desenrolado, 412-413
 - leitura atrasada, 412
- MIPS R4000, microprocessador, 417-419
- múltiplos fluxos, 372-373
- operações vetoriais, 543-546
- otimização, 411-413
- previsão de desvio, 374-377
- processador Cortex-A8, 450, 452
- Single-instruction multiple-data (SIMD)*, instruções, 452
- técnica superescalar comparada com, 432
- técnica superpipeline, 432
- Pistas, compact disk, 166
- Pistas, discos compactos, 166
- Placas, 150, 153-154
- Política de escrita, memória cache, 110-111
- Ponteiro da próxima instrução na cache de trace, processador Pentium 4, 443-444
- Ponto flutuante, notação, 267-280, 446, 452
 - adição, 272-275
 - aritmética, 272-280
 - arredondamento, 277-279
 - bits de guarda, 276-277
 - considerações de precisão, 276-279
 - dados da unidade aritmética e lógica (ALU), 267-280
 - divisão, 275-279
 - interpretação de infinito, 279
 - multiplicação, 275-279
 - NaNs, 279
 - números desnormalizados, 279-280
 - números normalizados, 269-270
 - overflow*, 270, 273
 - padrões IEEE para, 271-272, 277-280
 - pipeline do processador Cortex-A8, 452-453
 - princípios, 267-271
 - representação polarizada, 268
 - representação, 267-272
 - significando, 268, 273
 - subtração, 273-275
 - underflow*, 270, 273
 - unidade de execução do Pentium 4, 446
 - valor de expoente, 268, 273
- Pós-indexação, 334, 338
- Processador matricial, 542, 546
- Processadores multicore, 33-34
- Processadores, 9, 68, 179-180, 248, 355-394
 - Advanced RISC machine (ARM)*, organização, 378-389
 - ciclo da instrução, 361-364
 - comunicação, 68, 180
 - estrutura e função, 248, 355-394
 - Intel 8085, 472-475
 - Intel x86, organização, 378-385
 - interconexão do sistema (barramento), 10, 68, 356
 - modos, ARM, 386-387
 - módulos de E/S, 68, 180
 - organização interna, 470-471
 - pipeline de instruções, 364-378
 - processamento de interrupção, 382-383, 389
 - registradores, 11, 357-361, 378-382, 387-389

- requisitos funcionais, 466-467
- requisitos, 356-357
- sinais de controle, 468-469, 473
- sinais, 68
- unidade aritmética e lógica (ALU), 11, 356
- unidade de controle, 11, 466-475
- Processadores superescalares, 248, 396, 429-458
 - Advanced RISC machine* (ARM) Cortex-A8, 446-452
 - concluir (*commit*) ou retirar instruções, 440
 - conclusão em-ordem, 435
 - conclusão fora-de-ordem*, 436-437
 - desenvolvimento, 429-430
 - execução de programas, 440
 - implementação de programas, 441
 - Intel Pentium 4, 441-446
 - limitações do paralelismo, 432-434, 436-437
 - paralelismo em nível de instrução, 248, 429-458
 - política de emissão de instrução, 434-437
 - previsão de desvio, 439
 - questões de projeto, 434-437
 - renomeação de registradores, 437-438
 - sistemas CISC e RISC comparados com, 396
 - técnica de *superpipeline* comparada com, 430-432
- Processo, 219-224, 528-529
 - bloco de controle, 221
 - chaveamento (ou troca), 528
 - conceito, 219
 - escalonamento, 219-224, 528
 - estados, 220-222
 - execução, 221-223, 528
 - interrupção, 223
 - multithreading*, 528-529
 - posse do recurso, 528
- Processo de posse de recurso, 528
- Programa *hardwired*, 54
- Programas de *benchmark*, 41-44
- Programmable read-only memory* (PROM), 129, 131
- Projeto do formato de instrução do VAX, 344-346
- Projeto dos SO para multiprocessadores, considerações sobre, 520-521
- Proteção da memória, SO, 215
- Protocolos de diretório, 524
- Protocolos de *snoopy*, coerência de cache, 524-525
- PUSH*, operação de pilha, 322
- Q**
 - Quick Path Interconnect* (QPI), 571
 - Quiet NaN, 279
 - Quociente multiplicador (*Multiplier Quotient* – MQ), 16
- R**
 - RAID. Ver *Redundant Array of Independent Disks* (RAID)
 - Reduced Instruction Set Computers* (RISC), 1, 248, 395, 428
 - Arquitetura com conjunto reduzido de instruções, 405-410
 - arquitetura de processador escalável* (SPARC) *Scalable Processor Architecture*, 420-424
 - Característica do CISC *versus* RISC, 409-410
 - chamadas de procedimento, 399
 - CISC RISC e superescalares, 396
 - compiladores, 404-405
 - conjunto de instruções, 414-416, 421-423
 - desenvolvimento, 395
 - execuções de instruções, 396-400
 - formato da instrução, 408, 416, 423-424
 - instruções de máquina RISC, 407
 - linguagens de alto nível* (HLLs) *High-Level Languages* e, 396-400
 - MIPS R4000, 413-419
 - modos de endereçamento simples, 408
 - operações, 397-398
 - operandos, 398-399
 - Otimização de registradores baseada em Pipeline com instruções regulares, 410-413, 416-419
 - registrador-para-registrador, 407
 - registradores, 400-405, 420-421
 - Redundant array of independent disks* (RAID), 149, 157-164
 - características, 157-159
 - código de Hamming, via redundante (nível 2), 158-159, 162
 - espelhado (nível 1), 158-159, 161-162
 - intercalados* (*striped*), (nível 0), 159-161
 - não redundante (nível 0), 159-161
 - níveis, 157-159, 165
 - paridade distribuída em nível de bloco (nível 5), 158, 161, 164
 - paridade em nível de bloco (nível 4), 158, 161, 163-164
 - paridade por bit intercalado (nível 3), 158, 161, 162-163
 - redundância dual (nível 6), 158, 161, 164
 - redundância, 163
 - Registrador de buffer de instrução (*instruction buffer register* – IBR), 15
 - Registrador de buffer de memória (*memory buffer register* – MBR), 15, 55, 57, 359
 - Registrador de endereço de memória (*memory address register* – MAR), 15, 55, 57, 359
 - Registradores, 11, 15, 330-335, 357-361, 378-383, 387-389, 400-404, 420-421, 504, 548-550
 - Advanced RISC machine* (ARM), organização, 387-389
 - armazenamento de variável global, 402
 - buffer de instrução (IBR), 15-16, 359
 - buffer de memória (MBR), 15, 359
 - códigos de condicionais (*flags*), 358-359
 - contador de programa (PC), 15, 359
 - controle, 357, 359-360, 380-382
 - Current Program Status* (CPSR), 387-389
 - dados, 358
 - EFLAGS, 380-382
 - endereço, 358
 - endereço de memória (MAR), 15, 359
 - estado, 357, 359-361
 - IBM 3090, facilidade de vetor, 548-550
 - instrução (IR), 16, 359
 - janelas, 400-401, 420-421
 - memória cache em comparação, 402-404
 - memória do computador IAS, 15
 - microprocessador Intel 8086, 360-361
 - microprocessador Intel 80386, 360-361
 - microprocessador Motorola MC68000, 360-361
 - microsequeenciador TI 8800 SBD, 501-502
 - MMX, 382
 - modo de endereçamento, 330-335
 - modo de endereçamento indireto, 330, 332, 333
 - organização do Intel x86, 378-381
 - organizações de microprocessador, 360-361
 - otimização baseada em compiladores, 404-405
 - palavra de estado do programa (PSW), 359
 - propósito geral, 357, 387
 - Reduced Instruction Set Computers* (RISC), 400-405, 420-421
 - registradores, 11, 357-361
 - Scalable Processor Architecture* (SPARC), 420-421
 - uso de um banco grande, 400-404
 - visíveis ao usuário, 357-359
 - Registradores de controle, 357, 359-360, 380-382
 - Registradores de dados, 358
 - Registradores de endereço, 358
 - Registradores de propósito geral, 357, 387
 - Registradores de *status*, 357, 359-360
 - Registradores visíveis ao usuário, 357-359
 - Regra de Pollack, 562
 - Renomeação de registrador, 437-438, 446
 - Representação decimal agrupada, 292-293
 - Representação em sinal-magnitude, 251
 - Representação polarizada, 268-269
 - Requisição de interrupção, 190
 - Resistência e capacitância (RC), atraso, 32
 - Resto parcial, 266
 - RISC. Ver *Reduced Instruction Set Computers* (RISC)
 - Rotação (deslocamento cíclico), operação, 302
 - Rotational positional sensing* (RPS – detecção de posição rotacional), 155
 - Roubo de ciclo (*cycle stealing*), 192
- S**
 - Scalable Processor Architecture* (SPARC – arquitetura de processador escalável), 420-421
 - conjunto de instruções, 421-423
 - conjunto de registradores, 420-421
 - formato de instrução, 423-424
 - Segmentação, processador Pentium II, 233-235
 - Sensor magnetorresistivo, 150
 - Sequenciamento, 468, 487-491, 504-506
 - de microinstruções, 487-491

- geração de endereço, 489-491
- microsequenciador TI 8800 SDB, 504-505
- processador IBM 3033, 490-491
- processador LSI-11, 491
- técnicas de sequenciamento, 487-489
- unidade de controle, 468, 487, 492
- Sequenciamento de geração de endereço, 489-491
- Servidores *blade*, 537-538
- Setores, discos magnéticos, 151
- Signaling NaN*, 279
- Significando, 268, 273
- Simulação de campos contínuos, 541-542
- Sinais de controle, 178, 468-470, 473
 - entrada/saída (E/S), 178
 - Intel 8085, processador, 472-475
 - unidade de controle, 468-470
- Sinais de estado, E/S, 178
- Sinal de Grant (GNT), PCI, 82-83
- Sinal de requisição (REQ), PCI, 82-83
- Sincronização (temporização), 73-74, 87-88, 179-180, 474-475
 - assíncrona, 74
 - diagramas, 87-88
 - Intel 8085, processador, 474-475
 - interconexão de barramento, 73-74
 - módulos de E/S, 179-180
 - síncrona, 73
- Single Instruction, Single Data (SISD)*, dado, 515-516
- Single Large Expensive Disk (SLEP)*, 158
- Single-Instruction Multiple-Data (SIMD)*, 310-313, 451-452, 515-516
 - dado, 515-51
 - instruções de *pipelining*, 451-452
 - instruções Intel x86, 310-313
- Sistema de computação, 7-11, 51-246
 - barramento periférico (PCI), 76-83
 - diagramas de temporização, 87-88
 - entrada/saída (E/S), 52, 56, 176-209
 - estrutura de alto nível, 9-11, 51, 53-88
 - funções, 8-9, 53-66, 84-86
 - interconexões, 10-11, 51, 67-86
 - memória cache, 51, 89-127
 - memória externa, 52, 149-175
 - memória interna, 52, 128-148
 - suporte do sistema operacional (SO), 52, 210-246
- Sistema operacional (SO), 52, 210-246
 - Advanced RISC machine (ARM)*,
 - gerenciamento de memória, 238-243
 - batch*, 213
 - escalonamento, 210, 214, 219-224
 - evolução, 213-214
 - funções, 211-219
 - gerenciador de recursos, 212-213, 218-219
 - gerenciamento de memória, 210, 219, 224-243
 - instruções privilegiadas, 216
 - Intel Pentium II, gerenciamento de memória, 234-238
 - interativo, 213
 - interface usuário/computador, 211-212
 - interrupções, 216
 - multiprogramado, 213, 216-219
 - objetivos, 211-212
 - proteção da memória, 215
 - suporte ao sistema operacional, 52, 210-246
 - tempo compartilhado, 216, 219
 - tempo de preparação, 214
 - uniprogramação, 213, 219
 - utilitários, 211
- Sistema operacional (SO) em lote (*batch*), 213-219
 - Linguagem de controle de job (JCL - *job control language*), 215
 - monitor (residente), 214-216
 - multiprogramação, 216-219
- Sistema operacional (SO) interativo, 213
- Sistema operacional de multiprogramação (SO), 213, 216-219, 224-225
 - comparação com uniprogramação, 213, 224-225
 - gerenciamento de memória, 219, 224
 - lote, 216-219
- Sistemas embarcados, 35-36
- Sistemas de memória ópticos, 164-169
 - Blu-ray DVD*, 166, 169
 - Compact Disk (CD)*, 164-169
 - Digital Versatile Disk (DVD)*, 166, 168-169
 - discos ópticos de alta definição (HD DVD), 169
 - produtos, 166
- Sistemas de memória, 89-209
 - acerto, 93
 - acesso, 91, 94
 - cache, 89-127
 - capacidade, 90
 - características físicas, 92
 - desempenho, 91-92, 94, 121-127
 - dois níveis, 121-127
 - externos, 149-209
 - falha, 93
 - hierarquia, 92-95
 - internos, 128-148
 - localidade de referência, 93, 122-123
 - localização, 90
 - organização, 92
 - palavra, 90
 - secundária (auxiliar), 94
 - unidade de transferência, 90
 - unidades endereçáveis, 90
- Sistemas operacionais (SO) de tempo compartilhado, 219
- Sistemas operacionais de uniprogramação, 313
- Site Web, recursos, 3-4, 45
- Small Computer System Interface (SCSI)*, 199
- SMP, Ver multiprocessadores simétricos (SMP)
- Snoop Control Unit (SCU)* unidade de controle de monitoramento, 571, 573-574
- Sobreposição de busca, pipeline, 365, 373
- Software, 19, 54, 523-524, 563-567
 - aplicação executando em um sistema (*database scaling*), 563-565
 - coerência de cachê e protocolo, 523
 - componentes do computador, 54-55
 - desempenho de computador multicore, 563-567
 - implementação, 20
 - threading* de jogo da Valve, 565-567
 - Software Development Board (SBD)*, 501-509
 - Special Interest Group (SIG)*, PCI, 76
 - Static Random-Access Memory (SRAM)*, 130
 - Subações, *FireWire*, 202
 - Sub-redes, *InfiniBand*, 203
 - Subtração, 256-258, 273-275
 - números em (com) ponto flutuante, 273-275
 - operação de complemento de dois, 255-256
 - Sulcos, discos compactos, 166
 - Swapping*, gerenciamento de memória de E/S, 224-225
 - System Performance Evaluation Corporation (SPEC)*, 42-44
- T**
- Tabela de página, 228-229, 230-231, 239-240
- Target channel adapter (TCA* – adaptador do canal de destino), 203
- Tags*, memória cache, 95
- Taxa de execução de instrução, 39-41
- Taxa de transferência, 91-92
- Técnica de arbitração de barramento, E/S, 188
- Técnica de consulta por software (*poll software*), E/S, 188
- Técnica de memória não cacheável, 111
- Técnica de observação de barramento, 110
- Técnicas de sequenciamento lógico de controle de desvio, 487-489
- Técnica de transparência de hardware, 111
- Tempo de acesso (latência), 91, 155-156
- Tempo de busca, discos magnéticos, 155-156
- Tempo de ciclo de memória, 20-21, 91
- Tempo de ciclo pipeline, 368-369
- Tempo de transferência, discos magnéticos, 155-156
- Temporização assíncrona, 74-75
- Temporização síncrona, 73
- Texas Instruments. Ver TI 8800 *Software Development Board (SBD)*
- Thrashing*, 104, 230
- Thread*, 528
- Thread*, computadores multicore, 567
- Thumb, mistura de instruções do ARM, 349
- TI 8800 *Software Development Board (SBD)*, 501-511
 - campos de controle, 505-506
 - formato de microinstrução, 502-504
 - microsequenciador, 504-506
 - pilhas, 504
 - registradores, 504
 - unidade aritmética e lógica (ALU), 507-509
 - unidades de controle microprogramadas, 501-513
- Tipos de sistemas operacionais (SO), 213-214
- Transdutor de E/S, 178

Transferência de dados, 75-76, 80-82, 297-300
interconexão de barramento, 75-76
operações (*opcode*), 297-300
PCI, 80-82

Transistores, desenvolvimento dos, 19-21
Translation lookaside buffer (TLB), 232-233, 238
Transmissão de dados assíncrona, 201-202
Transmissão de dados isócrona, 201-202
Trilhas, discos magnéticos, 151, 153-154
Trocas de dados, 67-68

U

Último-a-entrar-primeiro-a-sair (*last in first out* – LIFO), lista, 321-322, 334-335
Ultra-large-scale integration (integração em escala ultragrande – ULSI), 26
underflow gradual, 280
Underflow, 270, 273, 280
Unidade aritmética e lógica (*arithmetic logic unit* – ALU), 11, 13, 247, 249-285, 507-509, 550-551
adição, 256-258, 272-275
considerações de precisão, 276-277
desenvolvimento, 19
divisão, 265-267, 275-279
funções do computador, 11, 249-250
IBM 3090, recurso vetorial, 550
inteiros, 250-267
notação de números em (ou com) ponto fixo, 255-267
multiplicação, 258-265, 275-279
notação de números complemento em dois, 251-253, 255-267
notação de números em (ou com) ponto flutuante, 267-280
subtração, 256-258, 272-275
TI 8800 SBD, 507-509
Unidade central de processamento – CPU, 9-11, 20-21, 55-56, 247-457
CISC (Computadores em conjuntos de instruções complexos), 396, 398, 424-425
conjuntos de instruções, 247-248, 286-354
desenvolvimento, 20-21
formatos de instrução, 248, 339-351
função e estrutura do processador, 9-11, 248, 355-394
funções componentes, 56
instrução de máquina, 286-293, 396-400, 407
modos de endereçamento, 248, 329-339
operações (*opcode*), 247, 287, 297-315, 397-398
operandos, 247, 287-288, 292-293, 398-399
ordenação *endian*, 325-328
paralelismo em nível de instrução, 248, 429-458
pilhas, 321-325
processadores superescalares, 248, 396, 429-457
registradores, 11, 356-361, 378-382, 387-389

RISC (conjunto de instruções reduzido), 248, 395-428
tipos de dados, 294-296
unidade aritmética e lógica – ALU, 11, 247, 249-285

Unidade de busca de instrução, processador Cortex-A8, 447-449

Unidade de controle, 11, 396, 459-511
controle do processador, 11, 466-475
entradas, 475-476
execução, 468, 492-501
implementação hardware, 475-477, 485
Intel 8085, processador, 472-475
organização interna do processador, 470-472
lógica, 476-477
microinstruções, 480-482, 501-509
micro-operações, 462-466
microprogramada, 396, 459-460, 479-511
operação, 459, 461-478
requisitos funcionais, 466-468
sequenciamento, 468, 487-491
sinais de controle, 468-470, 473

Unidade de controle hardware, 475-477, 485

Unidade de controle térmica, 568

Unidade de decodificação de instrução, processador Cortex-A8, 449-450

Unidade de disco, E/S, 179

Unidade de gerenciamento de memória (*memory management unit* – MMU), 98, 239

Unidade de transferência, 91

Unidades de controle microprogramadas, 396, 459-460, 479-611
decodificadores, 484
desenvolvimento, 479-480
execução, 492-501
firmware, 480
LSI-11, processador, 491, 496-499
memória de controle, 481
microarquitetura, 482-483
microinstruções, 480-482, 487-510
processador IBM 3033, processador, 489-491, 499-501
sequenciamento, 487-491
TI 8800 *Software Development Board* (SBD), 501-509
vantagens e desvantagens, 486-487
Wilkes, exemplo de, 484-486

Unidades endereçáveis, 90

Uniform memory access (acesso uniforme à memória – UMA), 539

Universal Automatic Computer (UNIVAC), 19

Usado menos frequentemente (*least frequently used* – LFU), algoritmo, 110

Usado menos recentemente (*least recently used* – LRU), algoritmo, 110, 230

USENET, *newsgroups*, 4

Utilitários do SO, 211-212

V

Valor de expoente, 268, 273

Valve, software de jogo da, 565-567

Válvulas, desenvolvimento, 12-19

Vector Floating-Point (VFP – unidade vetorial de ponto flutuante), 571

Velocidade Angular Constante (*Constant angular velocity* – CAV), 151, 166

Velocidade de *clock*, 39-41

Velocidade Linear Constante (*Constant linear velocity* – CLV), 167

Very-Large-Scale Integration (VLSI – integração em escala muito grande), 26

Virgula fracionada, 250

Von Neuman, máquina, 13-19, 54-56

W

WAR (dependência de escrita após leitura), 432-434

Watchdog, 571

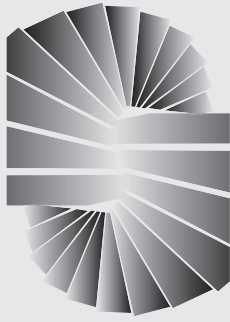
WAW (dependência de escrita após escrita), 436

Wilkes, controle, 484-487

Winchester, formato de disco, 152, 154

Write back, técnica, 110, 523

Write through, técnica, 110, 523



Sobre o autor

William Stallings tem contribuído de maneira singular para a compreensão da grande gama de desenvolvimentos técnicos em segurança, redes e arquitetura de computadores: é autor de 17 títulos e, contando com as diversas edições revisadas, tem publicados 42 livros que abordam diferentes aspectos de tais disciplinas. Seus escritos fizeram parte de publicações da ACM e IEEE, incluindo *Proceedings of the IEEE* e *ACM Computing Reviews*, e ele já recebeu 10 vezes o prêmio de melhor livro-texto de ciência da computação do ano da *Text and Academic Authors Association*.

Com mais de 30 anos na área, Stallings foi colaborador técnico, gerente técnico e executivo de várias empresas de alta tecnologia. Ele projetou e implementou conjuntos de protocolos baseados em TCP/IP e OSI em diversos computadores e sistemas operacionais, variando desde microcomputadores até mainframes. Como consultor, assessorou agências do governo, fabricantes de computadores e softwares, e importantes usuários em questões relacionadas a projeto, seleção e uso de software e produtos de rede.

O autor criou e mantém o *Computer Science Student Resource Site*, em <WilliamStallings.com/StudentSupport.html>. Esse site oferece documentos e links sobre assuntos de interesse geral aos alunos (e profissionais) de ciência da computação. Ele é membro da redação do *Cryptologia*, um jornal dedicado aos diversos aspectos que envolvem a criptologia.

Dr. Stallings possui PhD pelo M.I.T. em ciência da computação e B.S. pela Universidade de Notre Dame em engenharia elétrica.

WILLIAM STALLINGS

**ARQUITETURA E ORGANIZAÇÃO
DE COMPUTADORES** 8ª edição

Quatro vezes premiado como o melhor livro-texto de engenharia e ciências da computação pela Text and Academic Authors Association, *Arquitetura e organização de computadores* é bibliografia essencial a estudantes e profissionais da área.

Nesta oitava edição, William Stallings apresenta os fundamentos do processador e do design de computadores completamente atualizados e vai além, abordando questões relacionadas à memória, à E/S e a sistemas paralelos e trazendo exemplos concretos que auxiliam nas escolhas necessárias durante a implementação de um sistema operacional atual.

Acompanhando a rápida evolução da tecnologia, o conteúdo de *Arquitetura e organização de computadores* não se restringe a suas páginas: no site de apoio da obra, estudantes de ciências da computação, engenharia da computação e engenharia elétrica encontram capítulos e apêndices on-line (em inglês) e leituras complementares que ampliam seu conhecimento, bem como simuladores e exercícios que proporcionam a prática.



www.pearson.com.br/stallings

Professores e alunos encontram vasto material que complementa o conteúdo abordado no livro.

www.pearson.com.br

